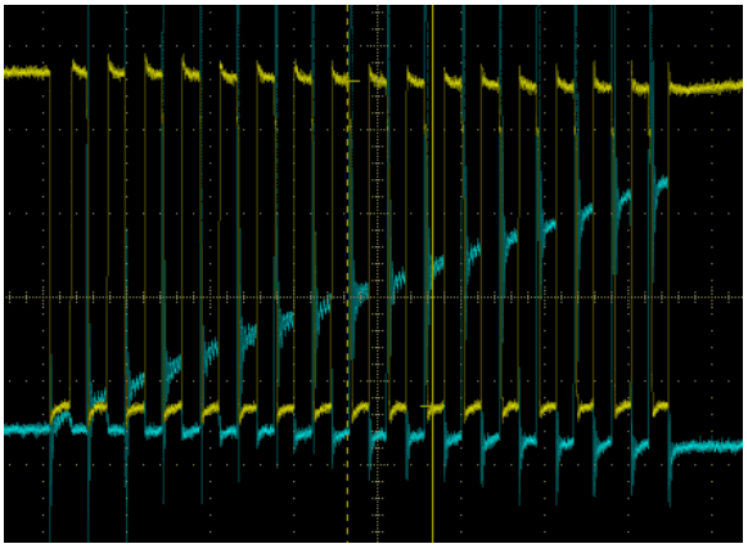
Gate driver and double pulse tests



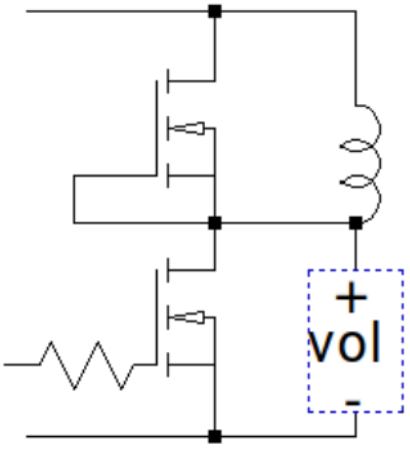


Table of contents

Primary driver

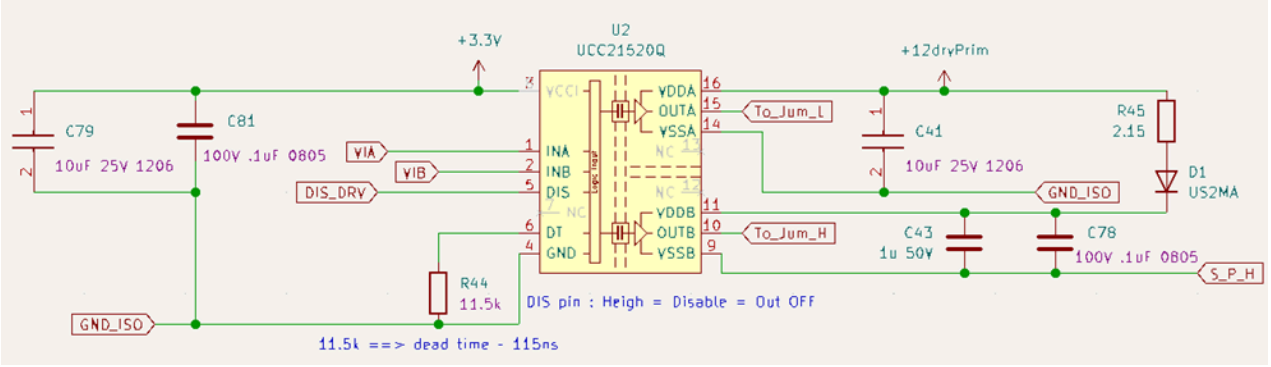


Figure 1 Reminder of the primary gate driver sub-schematic: Final version

Driver HW dead time

Let's check the accuracy of the programmed (HW) dead time of the used driver (UCC21520Q)

No SW dead time was programmed, the dead time is managed ONLY by thr RT = 11.5kΩ

Oscilloscope:

- **CH1** : INA
- **CH2** : INB
- **CH3** : OUTA

- **CH4** : OUTB

Driver:

ucc21520Q

Programmed dead time

RT = 11.5k, so dead time = 115ns

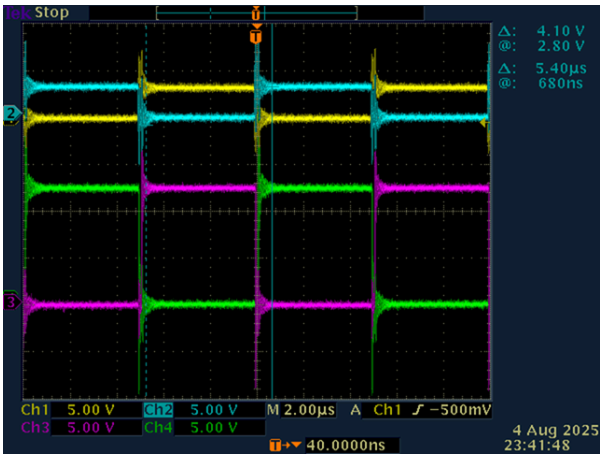


Figure 2 Zoom out to see 2 periods, 100khz

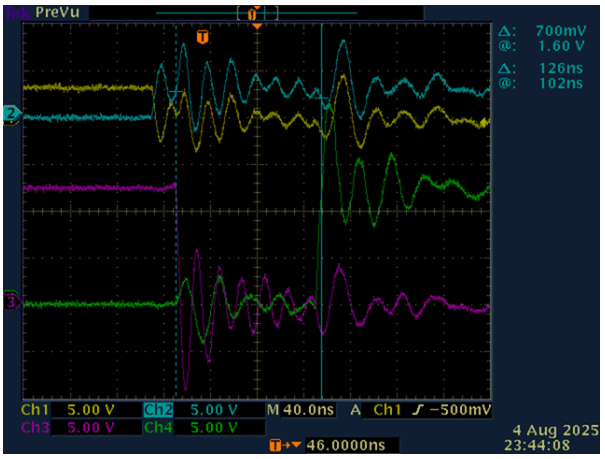


Figure 3 Zoom in around the dead time: DT=126ns

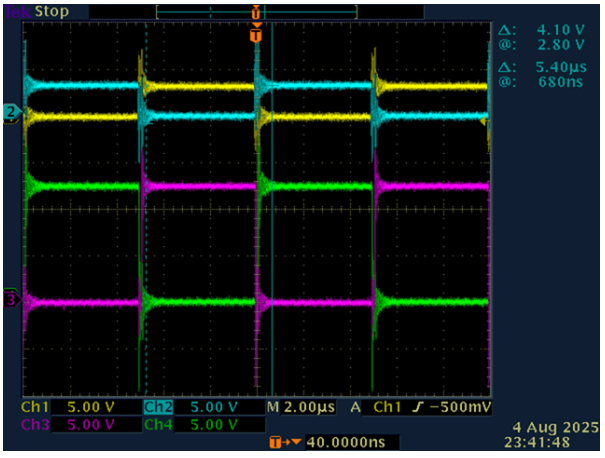


Figure 4 Zoom out to see 2 periods, 100khz

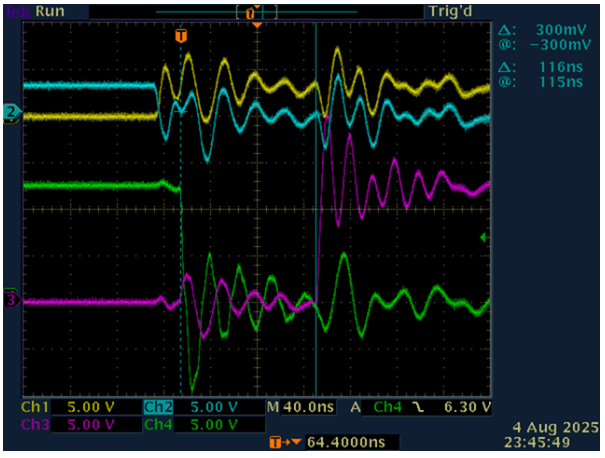


Figure 5 Zoom in around the dead time: DT=116ns

$$\mathrm{DT}_{Prog} = 115 \ \ (\mathrm{ns})$$
$$\mathrm{DT}_{Meas} = 116 \ \ (\mathrm{ns})$$
$$\mathrm{Error} = 100 \cdot \frac{\mathrm{DT}_{Meas} - \mathrm{DT}_{Prog}}{\mathrm{DT}_{Prog}} = 100 \cdot \frac{116 - 115}{115} \qquad = 0.87 \ \ (\backslash\%)$$

So the mesured dead time is very accurate

Bootstrap circuit

Lest's check the bootstrap circuit (DRC)

SINGLE pulse

In this test, the low-side MOSFET is driven by a single long pulse.

Peak current estimation :

$$\begin{aligned} \mathrm{V12} &= 12 \ \ (\mathrm{V}) \\ \mathrm{Vd} &= 0.700 \ \ (\mathrm{V}) \\ \mathrm{Rboot} &= 2.150 \\ \mathrm{Iboot} &= \frac{\mathrm{V12} - \mathrm{Vd}}{\mathrm{Rboot}} = \frac{12 - 0.700}{2.150} \qquad = 5.256 \end{aligned}$$

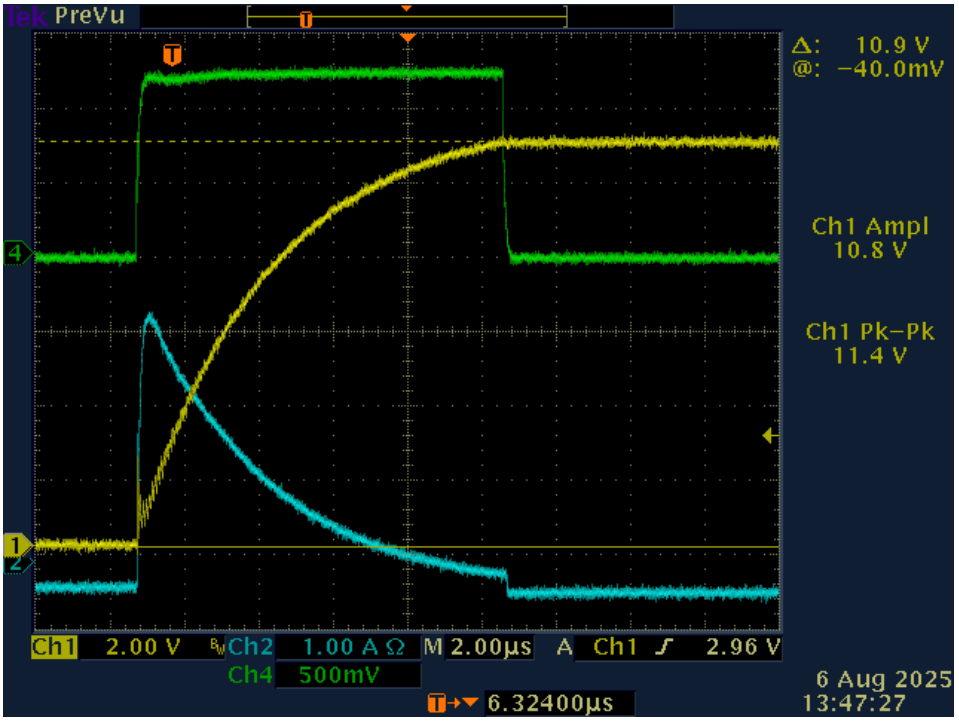


Figure 6 Bootstrap circuit measurement with 10us pulse width

Oscilloscope:

- **CH1** : bootstrap capacitor voltage
- **CH2** : bootstrap diode current
- **CH4** : Vgs low side MOSFET

Peak current measurement :

$I_{bootMeasure}$ ~ **3.8 Apeak**

The measured peak current is lower than the estimated value because the estimate was based on the assumption that the only resistance in the path was the 2.15-Ωresistor, which represents a pessimistic (worst-case) assumption.

Below is a fast/simplified reproduction of this test in simulation

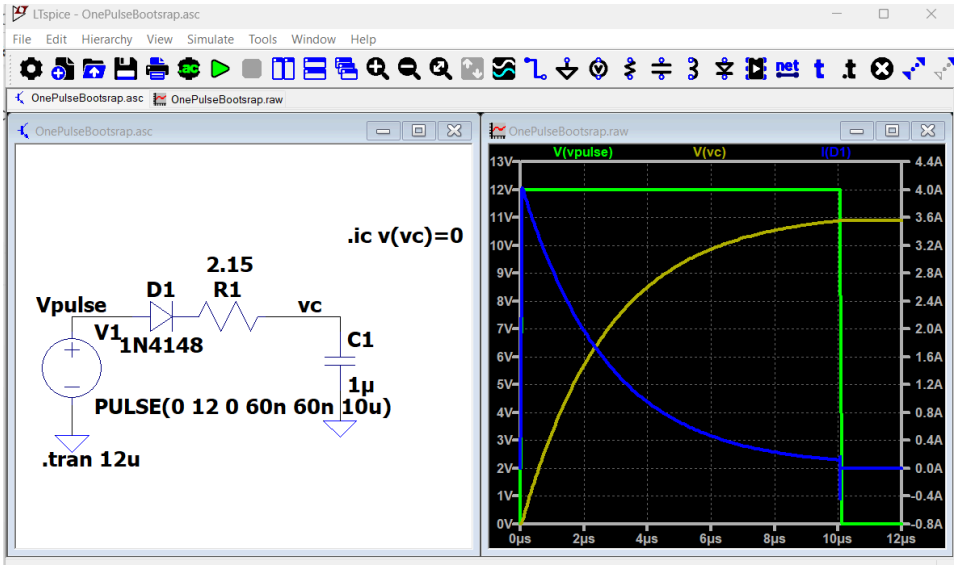


Figure 7 Quick simulation of the bootstrap circuit with 10us pulse width

PWM mode

In this test, the low-side MOSFET is driven by a pulse train with frequency of 100kHz (more realistic).

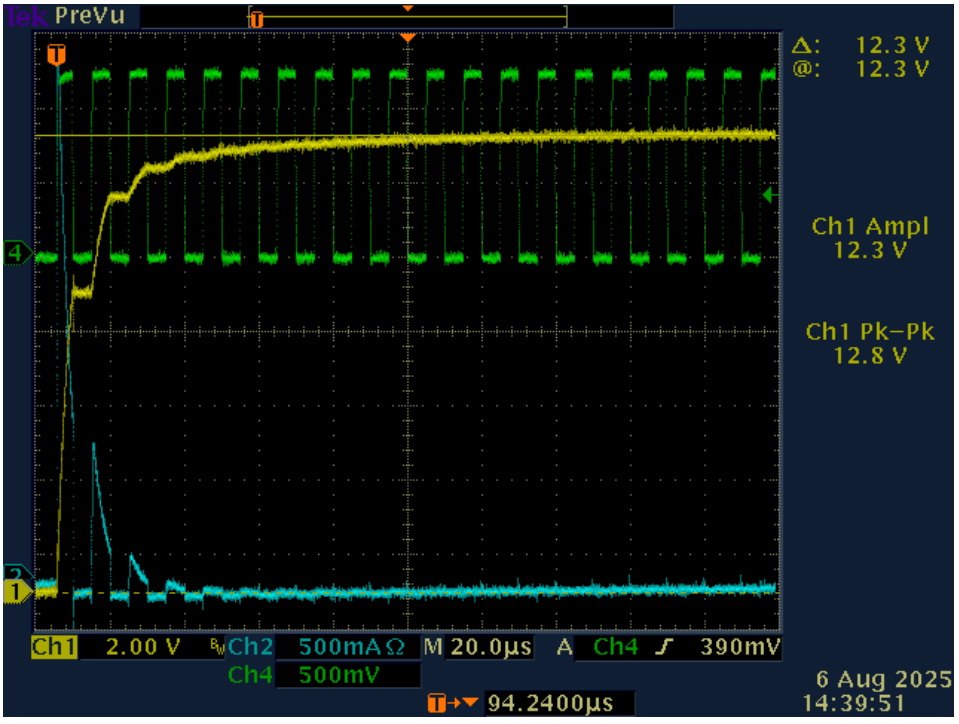


Figure 8 Bootstrap circuit measurement with PWM 100kHz 50% duty cycle

Below is a fast/simplified reproduction of this test in simulation

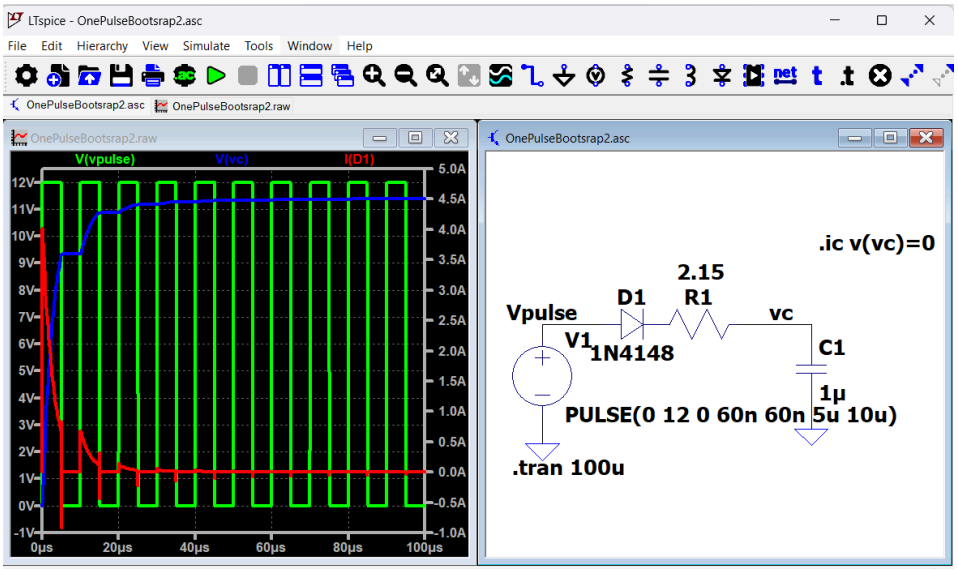


Figure 9 Quick simulation of the bootstrap circuit 100kHz 50% duty cycle

Estimation of the charge time of the bootstrap capacitor:

$$
\begin{aligned}
C &= 1 \ \ (\mu s) \\
R &= 2.150 \ \ (\text{ohm}) \\
\text{To} &= R \cdot C = 2.150 \cdot 1 \qquad = 2.150 \ \ (\mu s)
\end{aligned}
$$

3 corresponds to the 99% level, and 2 represents the 50% duty cycle.

$$
\text{time} = \text{To} \cdot 5 \cdot 2 = 2.150 \cdot 5 \cdot 2 \qquad = 21.500 \ \ (\mu s)
$$

So minimum time to charge the capacitor is 22µs or 3 periodes of 100khz

Double pulse test

Setup

We used the primary side of the DC-DC (without ferrite elements) to make the double pulse test.

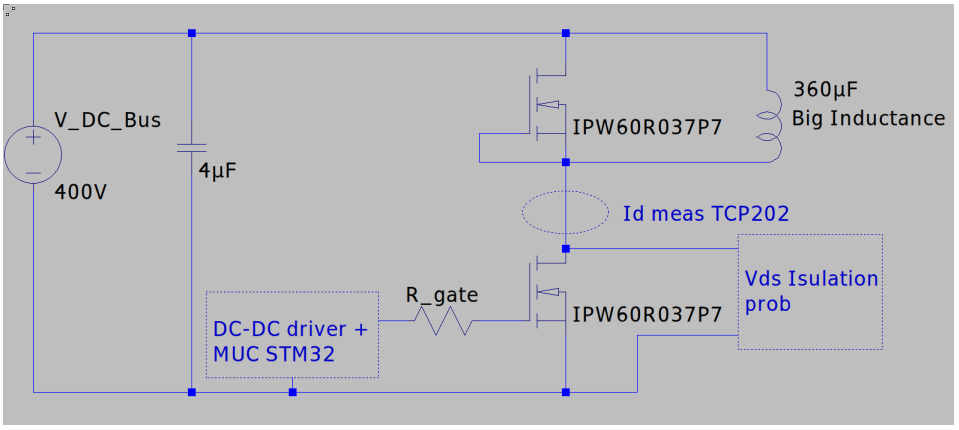


Figure 10 Double pulse setup

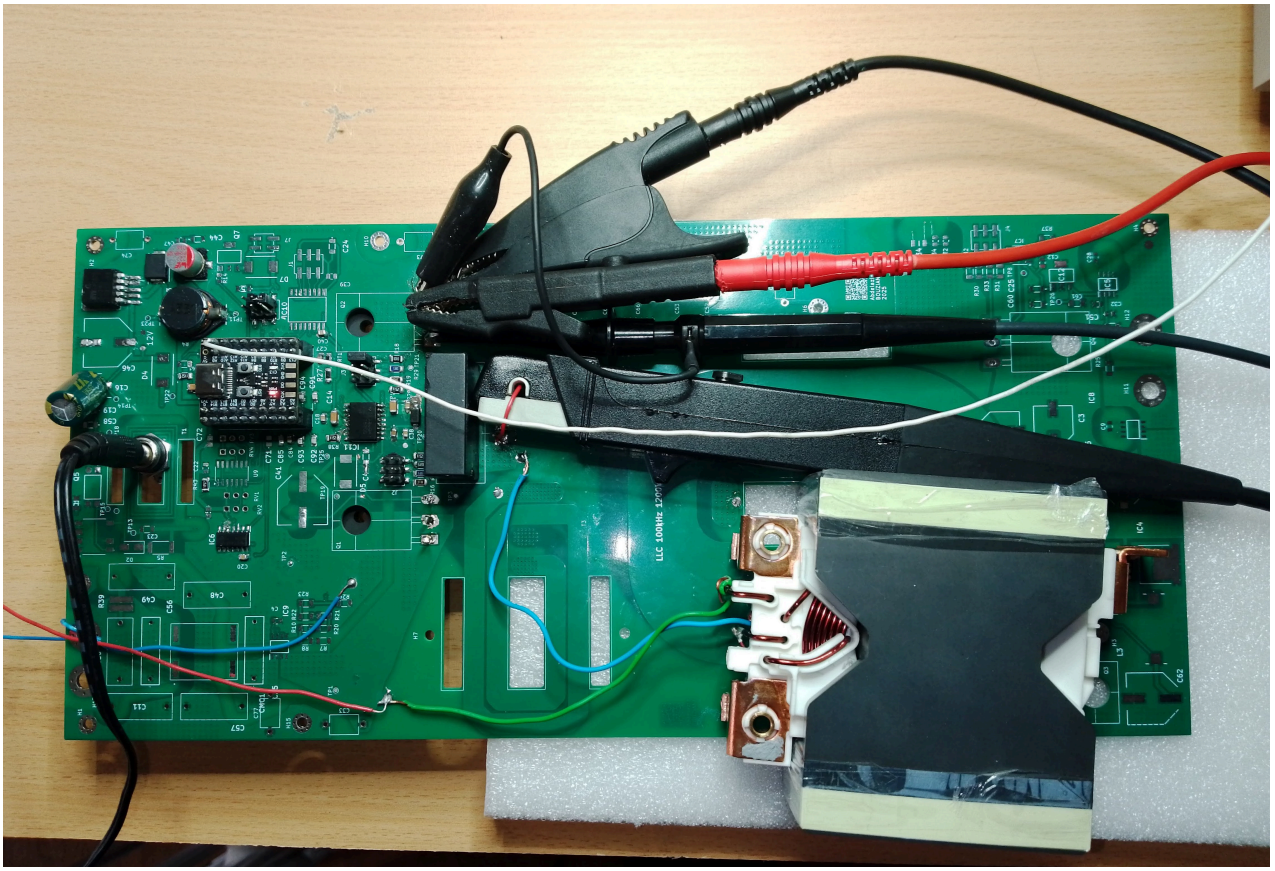


Figure 11 Double pulse setup (with the big inductance 360µH, 16 turns)

Example of the pre-processing

In this section an example of pre-processing measured signal to calculate the Eon, and Eoff energy

- V_bus = 366.5V
- V_lv = 12.34v
- R_gate_ext = 10Ω

Step 1: Load the csv files

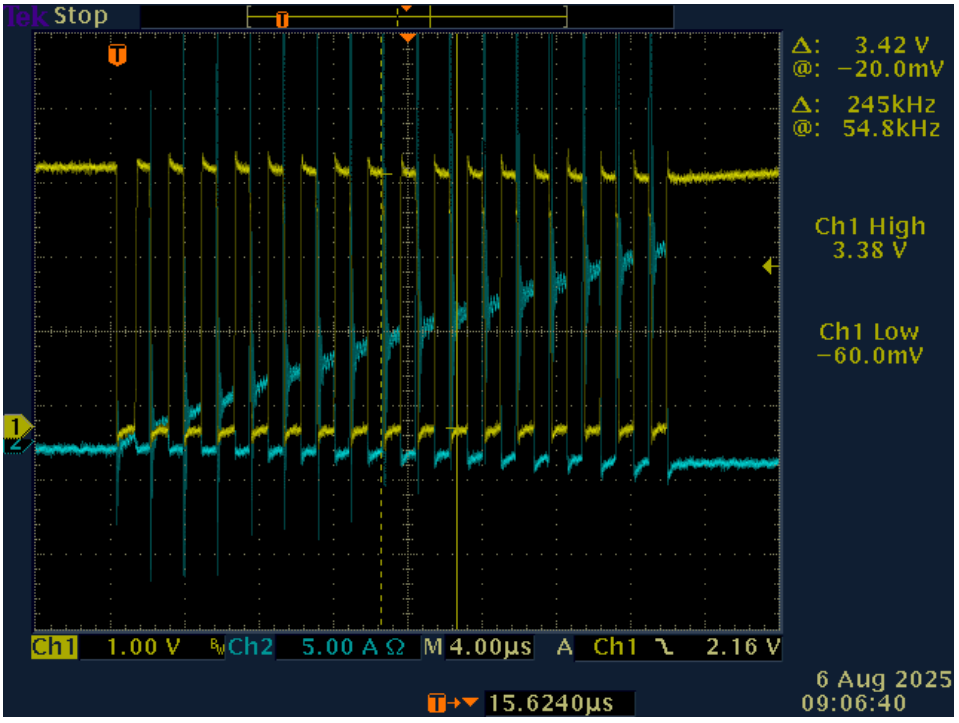
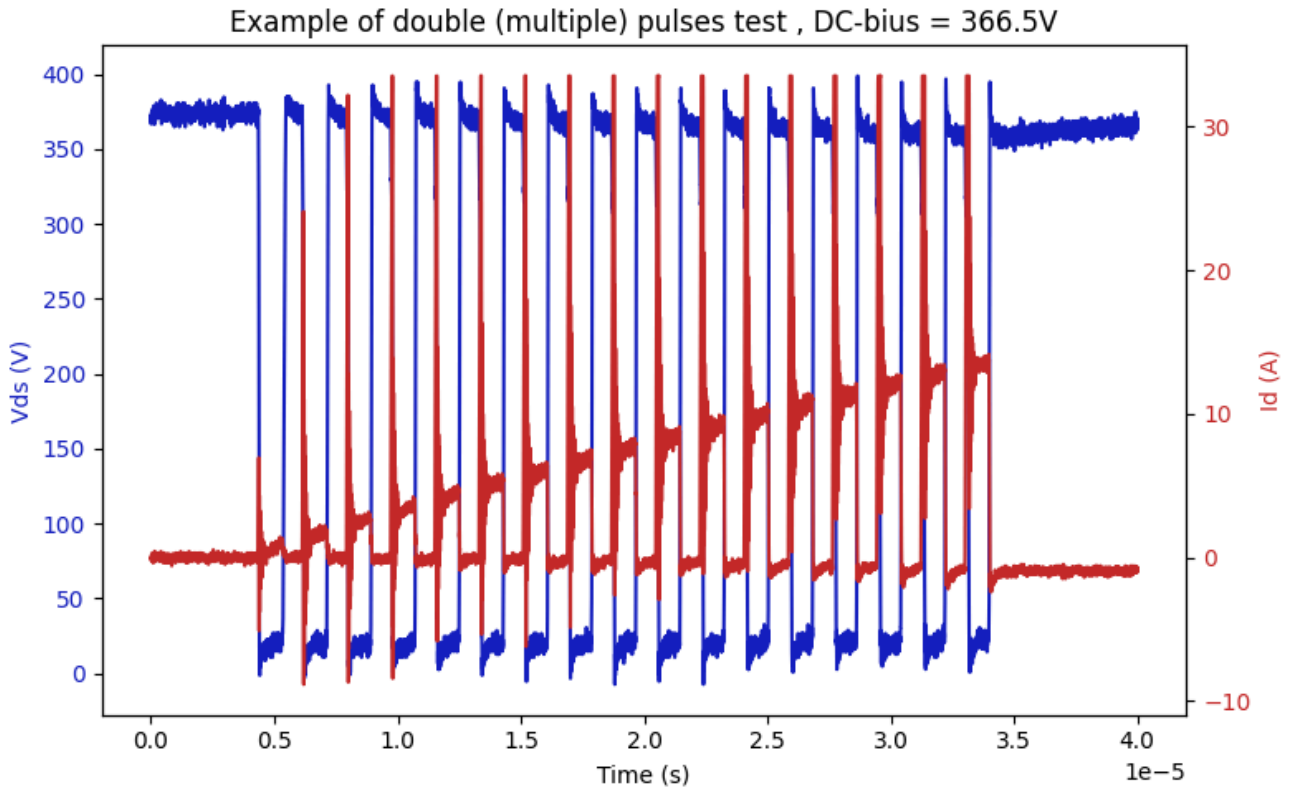
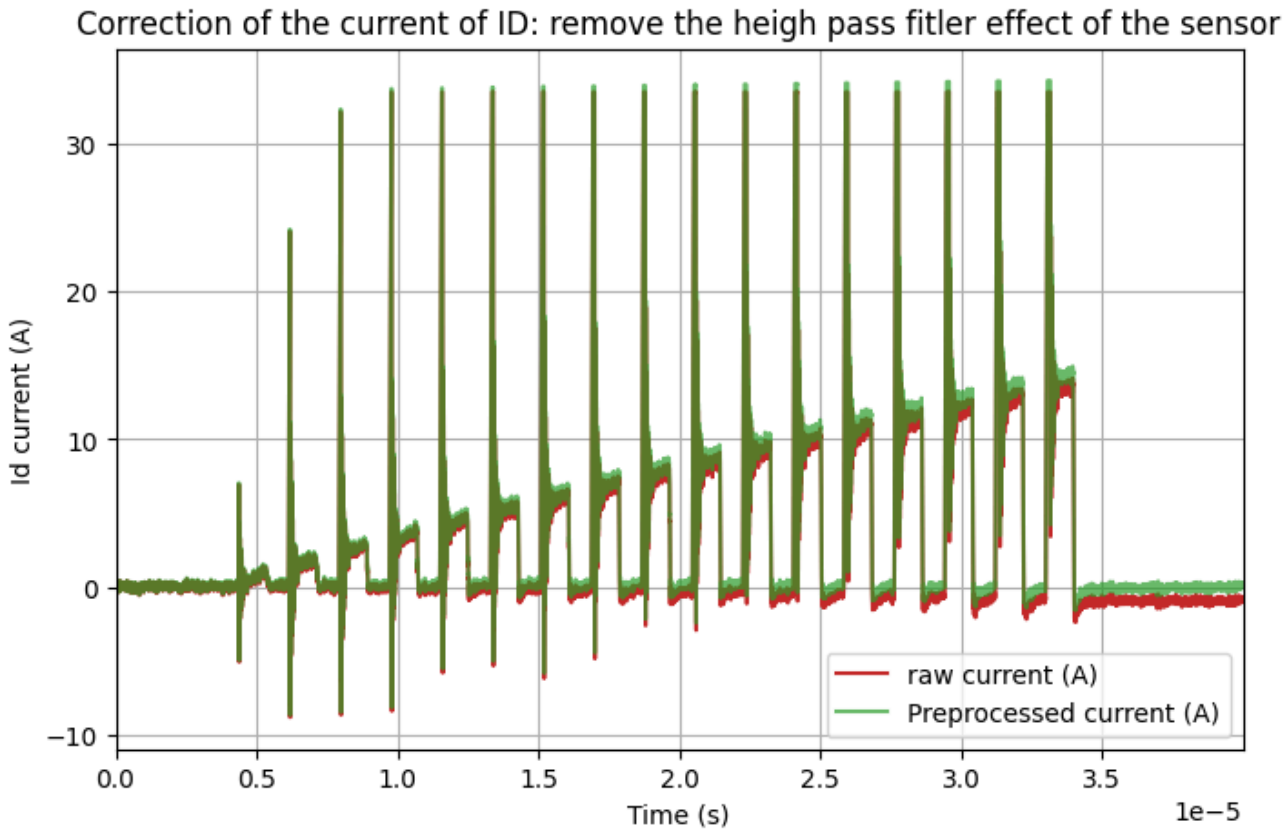


Figure 12 Quick simulation of the bootstrap circuit with 10us pulse width

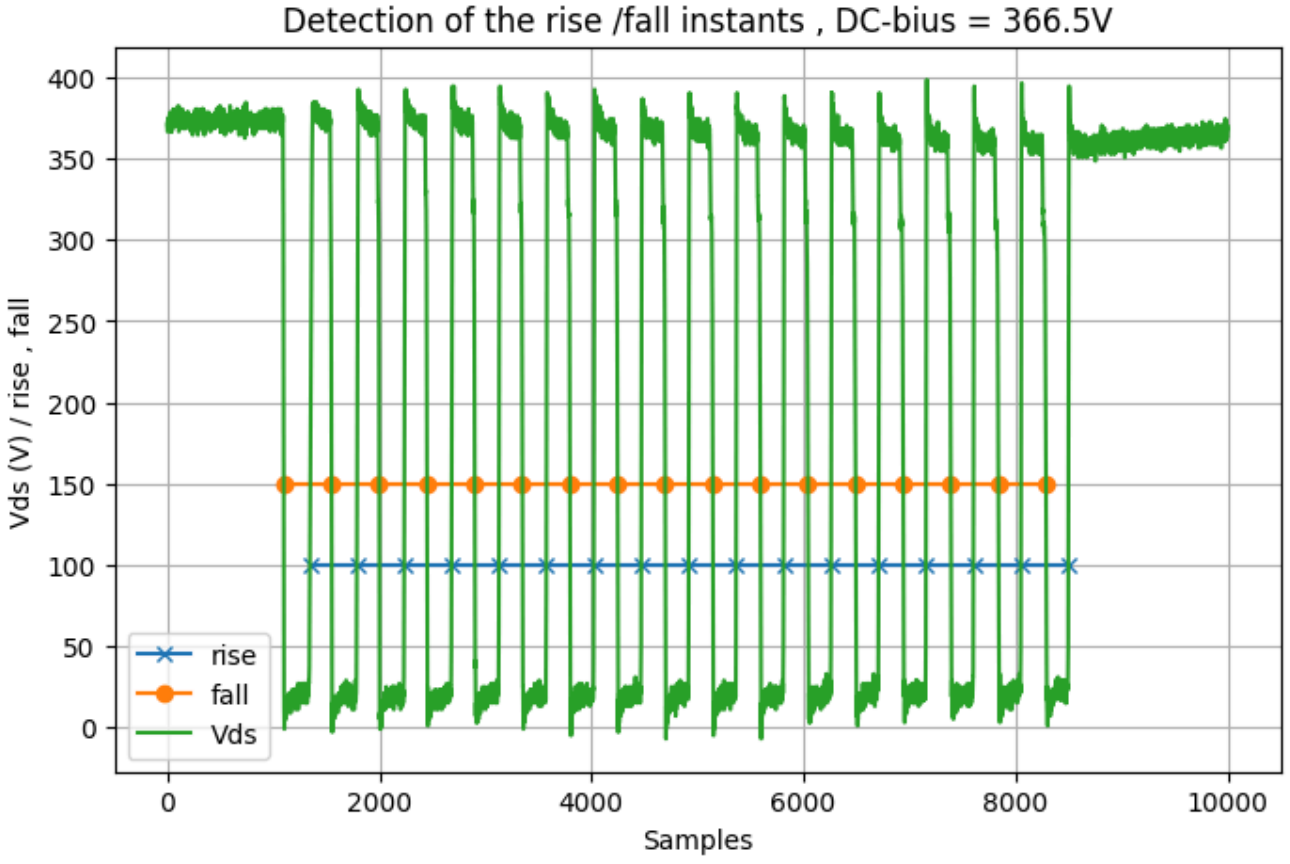


Step 2 : Pre-processing the current

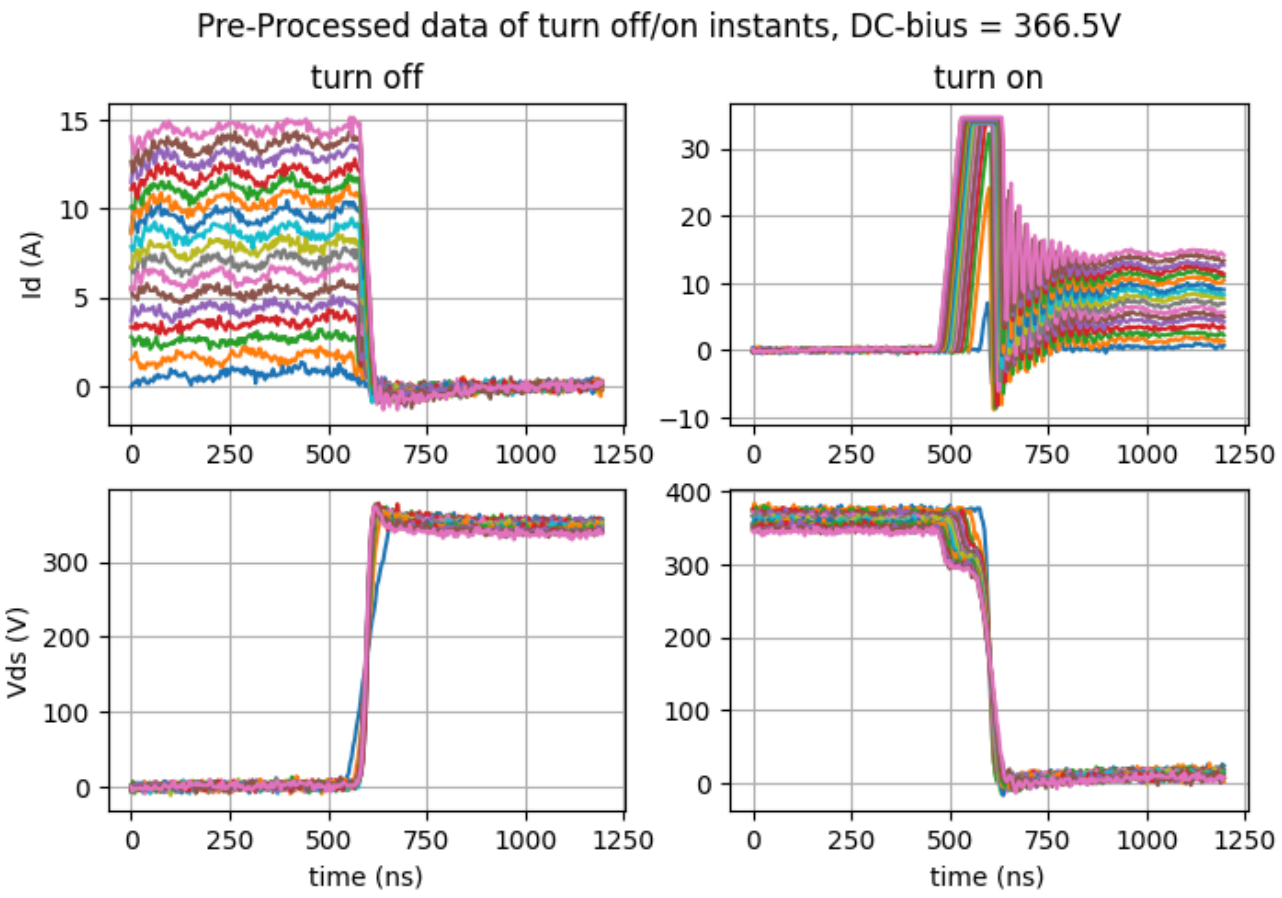
The used current prob has a heigh pass filter effect, so below a pre-processing to remove this effect.



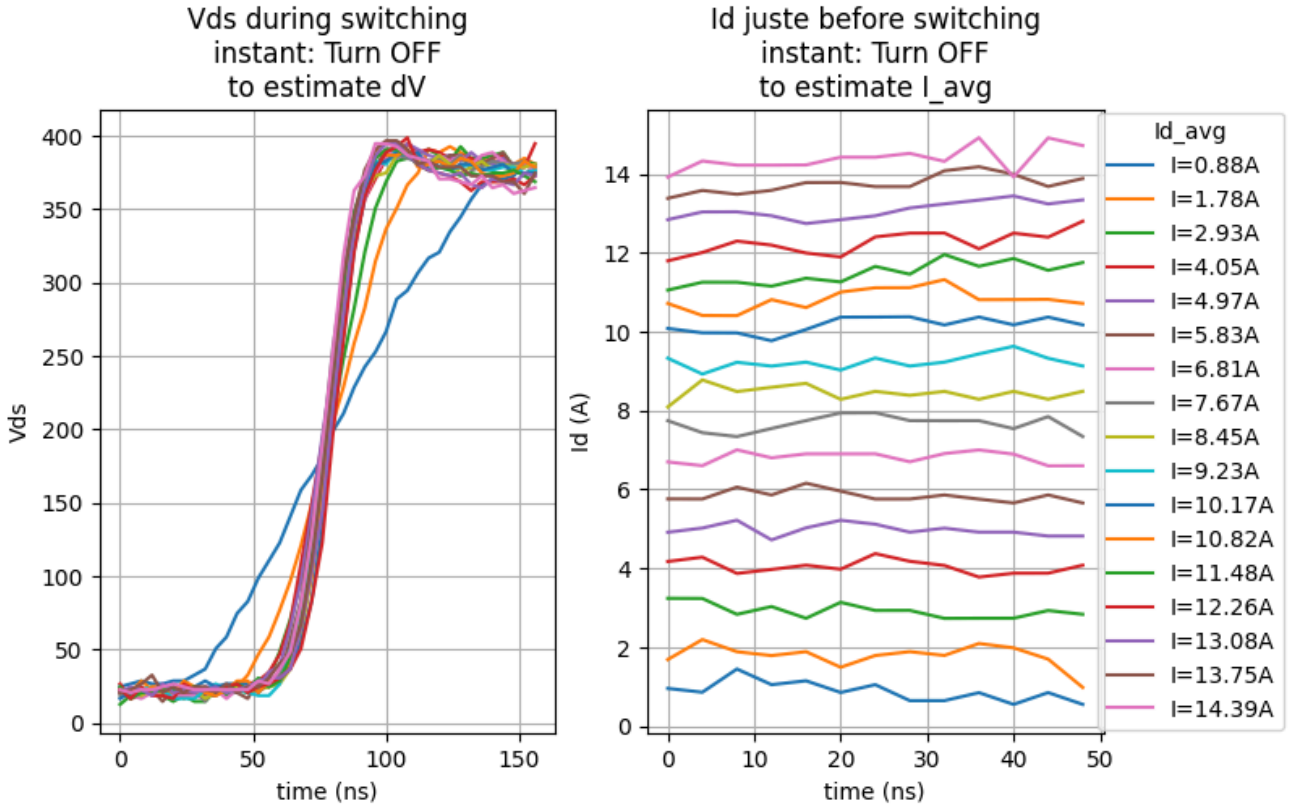
Step 3: Detection of the rise/fall instants



Step 4: zoom around the switching instants



Step 5: Estimation of the Coss capacitor



Formulas to calculate the mosfet capacitors:

Time related capacitance

$$i = C\frac{dv}{dt}$$

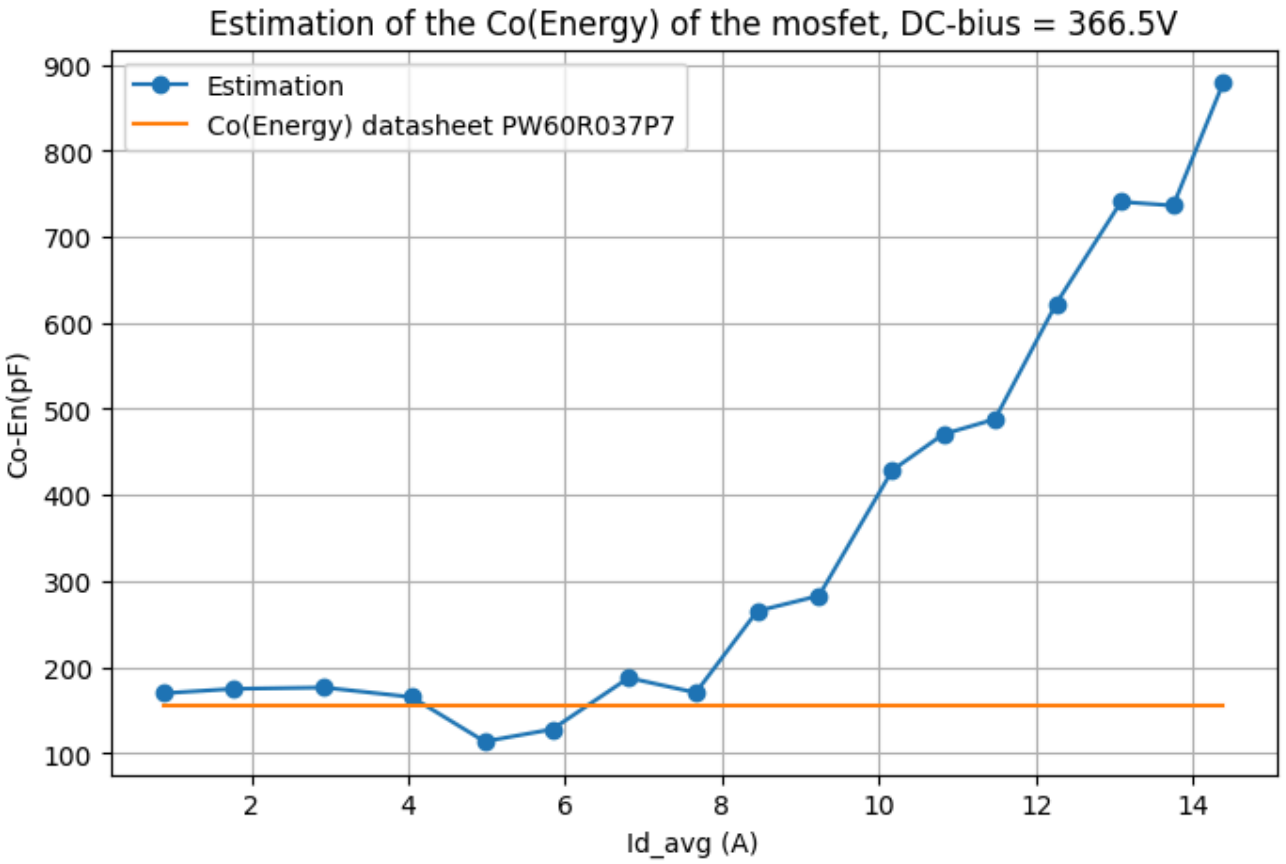
$$C_o(tr) = \frac{\int i \cdot dt}{\int dv} = \frac{Q}{\Delta V}$$

Energy-related capacitance

$$E = \int v \cdot i \cdot dt$$

$$C_{o(er)} = \frac{2E}{\Delta V^2}$$

dv is measured between min_v + 50V and max_v - 50V



Why the estimation of Co-Energy is good at low corrent, and bad in heigh current:

$$E_{total} = E_{oss} + E_{switching\_loss}$$

In low current loss energy is low so almost the all measured energy is the stored energy in the mosfet capacitor, but in heigh current a part of energy is converted to heat.

To reproduce this phynomina, I propose the simple simulation below:

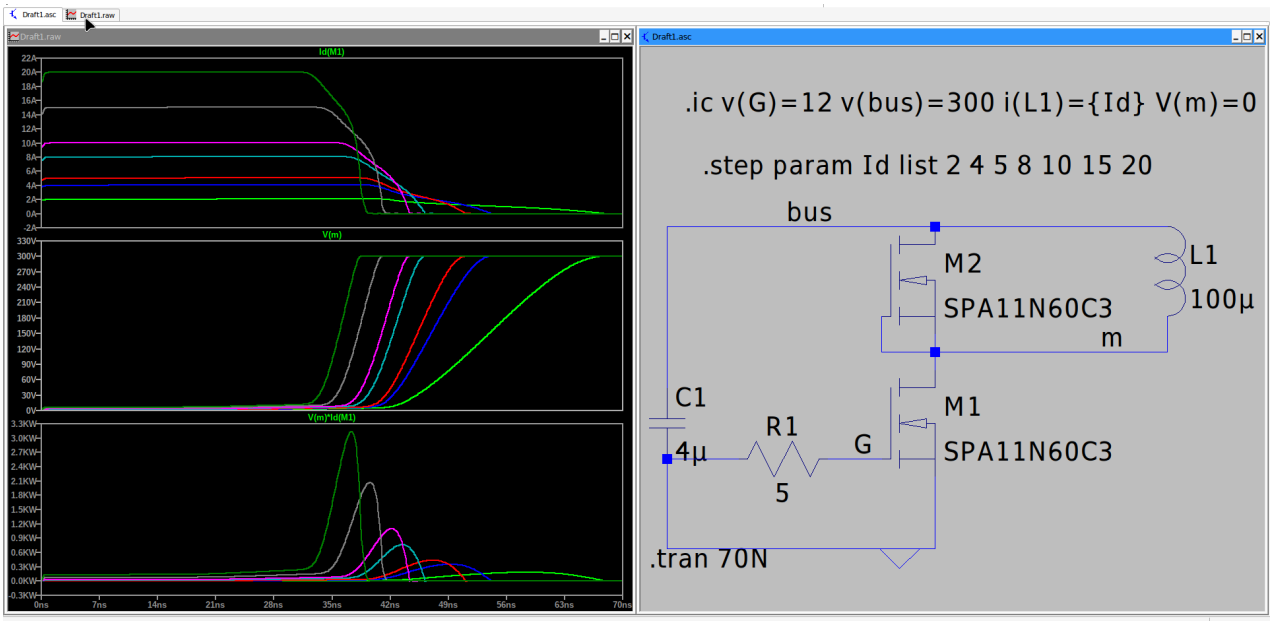
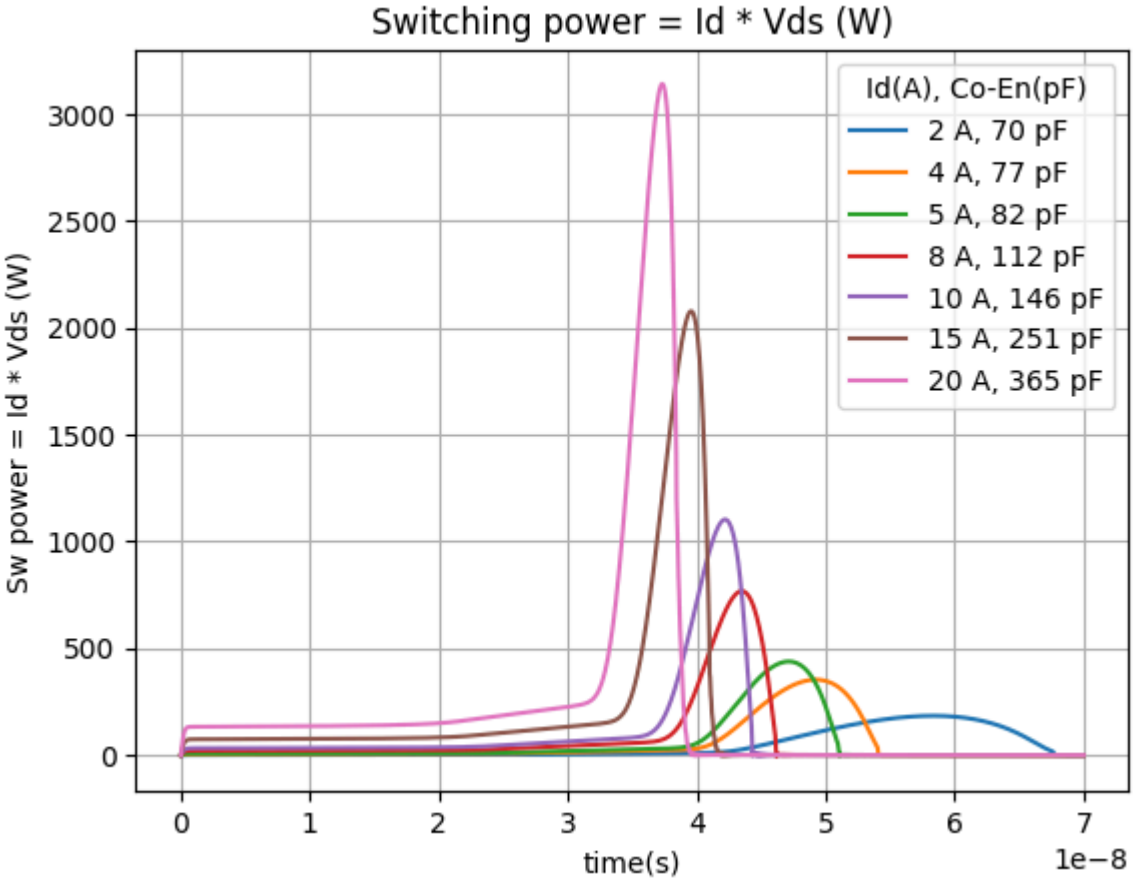
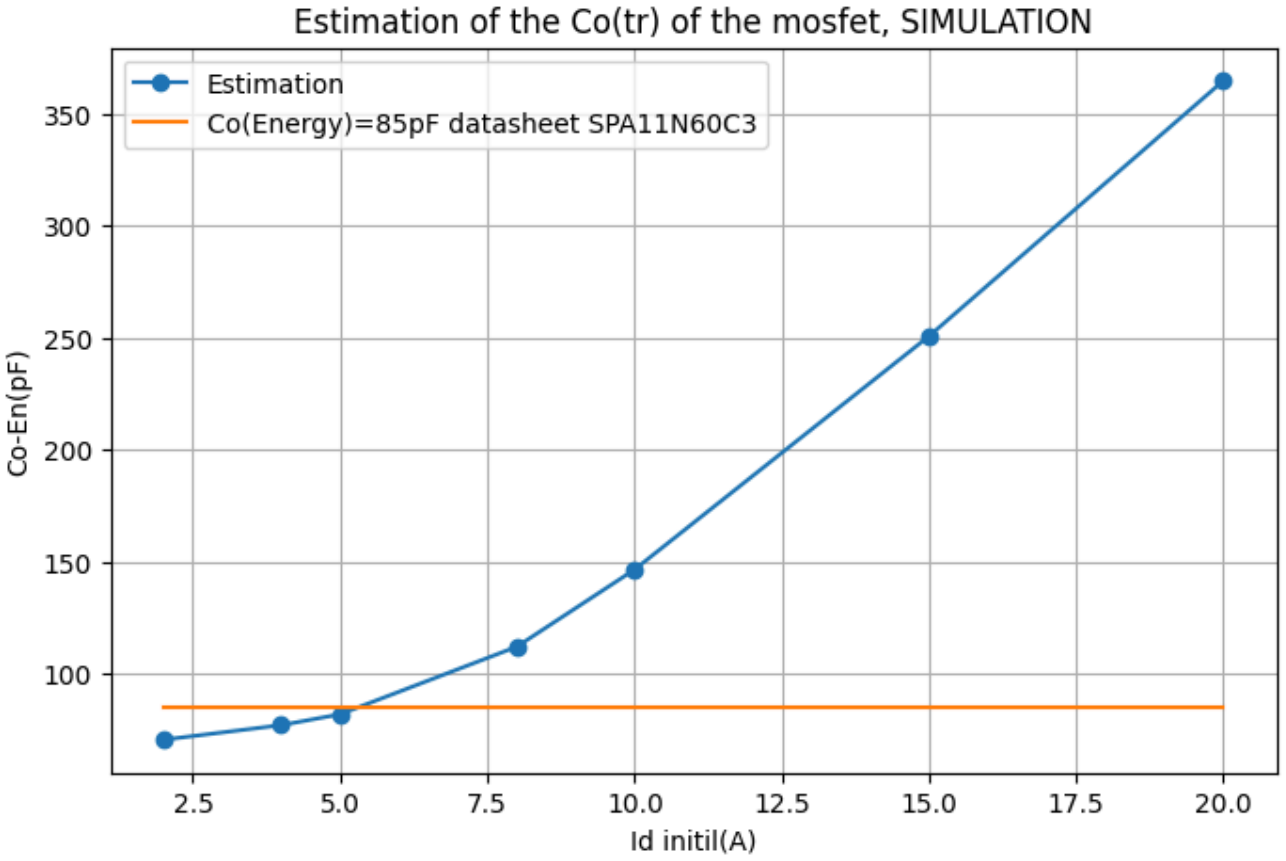


Figure 13 Simple simulation of the turning off instant of SPA11N60C3

Lest's calculating the sw-enrgy and the coresponding capacitor using pre-processing of the raw data using python

```
Text(0.5, 1.0, 'Switching power = Id * Vds (W)')
```

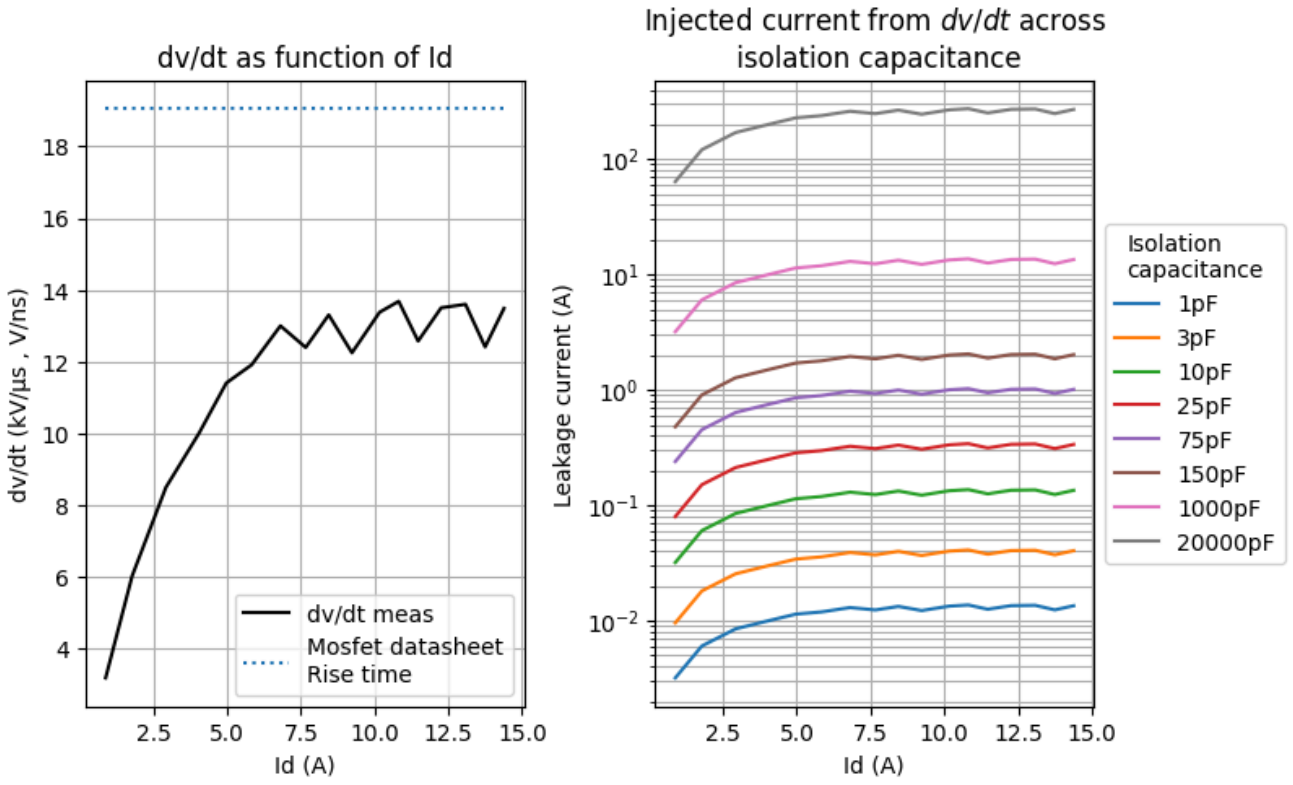




So , we can see the same behaviour our measurement

For information dv/dt:

Shown below are the dv/dt measured during turn-off and the corresponding current as a function of isolation capacitance, for the case where an isolated power supply is used to power the high-side driver.



In **IPW60R037P7** datasheet, we can cound :

- Rise time (tr) = 21ns
- @ VDD=400V,VGS=13V,ID=29.5A, RG=3.3Ω

So $dv/dt = 19V/ns$

The "dv/dt meas" has a maximum,this explain by

$$\frac{dv}{dt}_{max} = \frac{V_{gsPlateau} - V_{EE}}{R_{gtot} \cdot C_{gd}}$$

Since driver is powered by 12V/0V (no V_{EE}), so:

$$\frac{dv}{dt}_{max} = \frac{V_{gsPlateau}}{R_{gtot} \cdot C_{gd}}$$

where

C_{gd} : Gate-Drain capa, C_{rss} in datasheet

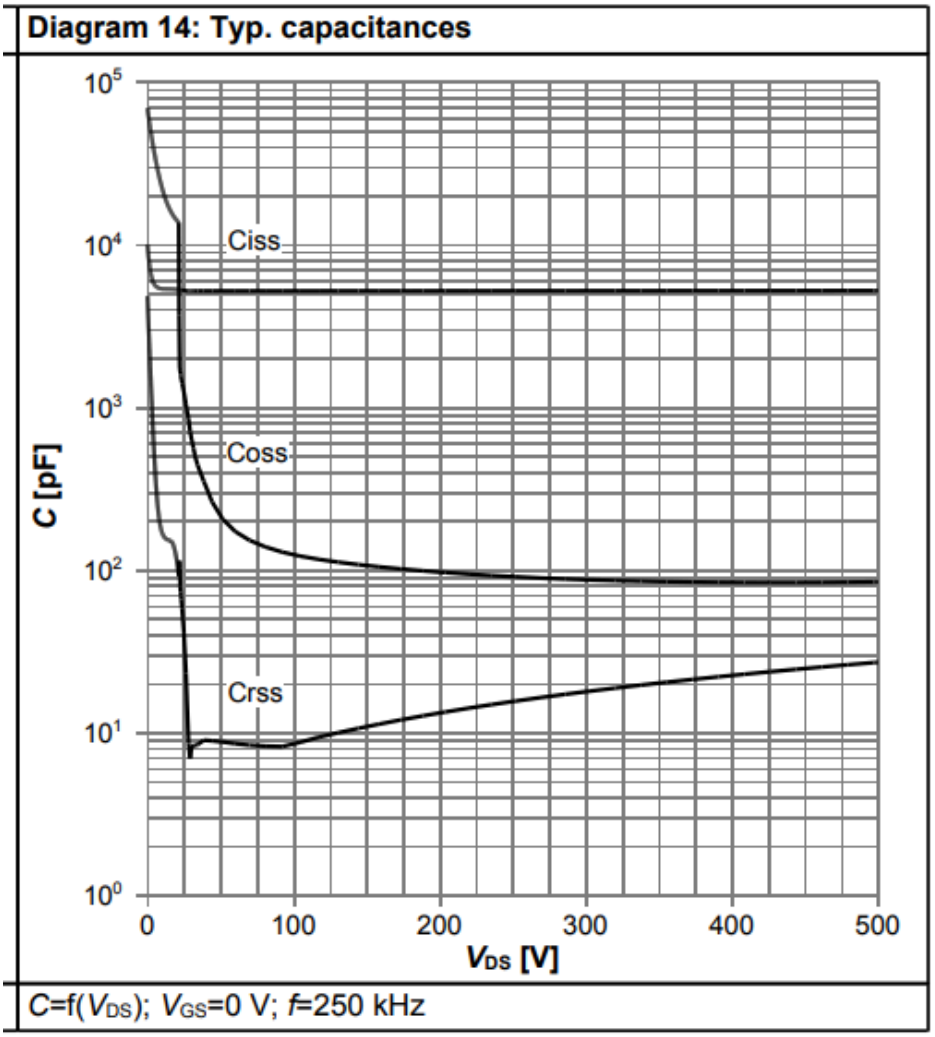


Figure 14 IPW60R037P7 datasheet: Crss = 22pF at 366V

Let's calculate the theorical dv/dt_max:

$$
\begin{aligned}
R_{gMosfet} &= 0.860 \ \ (\Omega \text{ , see IPW60R037P7 datasheet}) \\
R_{driver} &= 5 \ \ (\Omega \text{ , see UCC21520Q datasheet}) \\
R_{ext} &= 10 \ \ (\Omega \text{ , in this example}) \\
R_{gTot} &= R_{gMosfet} + R_{driver} + R_{ext} = 0.860 + 5 + 10 \qquad = 15.860 \ \ (\Omega)
\end{aligned}
$$

$$
\begin{aligned}
V_{gsPlateau} &= 5.200 \ \ (\text{V , see IPW60R037P7 datasheet}) \\
C_g &= 22 \ \ (\text{pF , see fig above}) \\
\text{dvdt} &= \frac{V_{gsPlateau}}{R_{gTot} \cdot C_g \cdot 1 \times 10^{-3}} = \frac{5.200}{15.860 \cdot 22 \cdot 1 \times 10^{-3}} \qquad = 14.903 \ \ (\text{V/ns})
\end{aligned}
$$

The maximum dv/dt measured is around 13 - 14 V/ns, so our theorical estimation is very close

Rg impact

In this section, we will change gate resistor, and check the impact in the loss energy.

We calculating the Eon just for information, since we are in ZVS, Eon is very low, and the calculated Eon is compatible with no ZVS function.

In the PCB we mount 3 resistors in parallel for Rg, 3 Ω , 5 Ω and 10 Ω with a jumper, so it will be easy to change Rg using this value, or any combinason of parallel configuration like 3 Ω // 5 Ω = 1.875 Ω and 3 Ω // 10 Ω =2.30 Ω (see the photo below)

Just for information, we broked the mosfet during this test @400V, so we recommande to no make SW-ON.

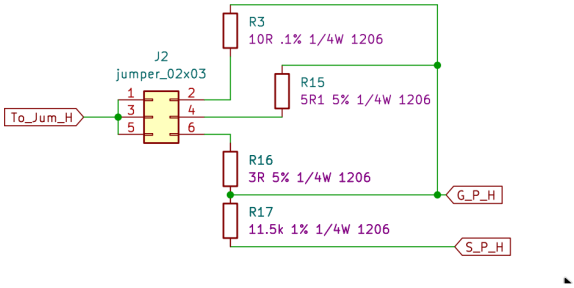


Figure 15 The Jumper used to select Rg

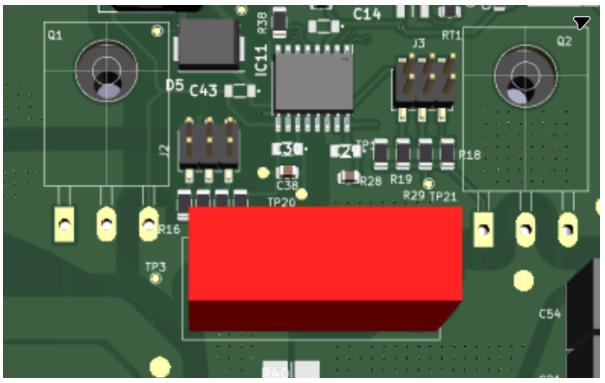
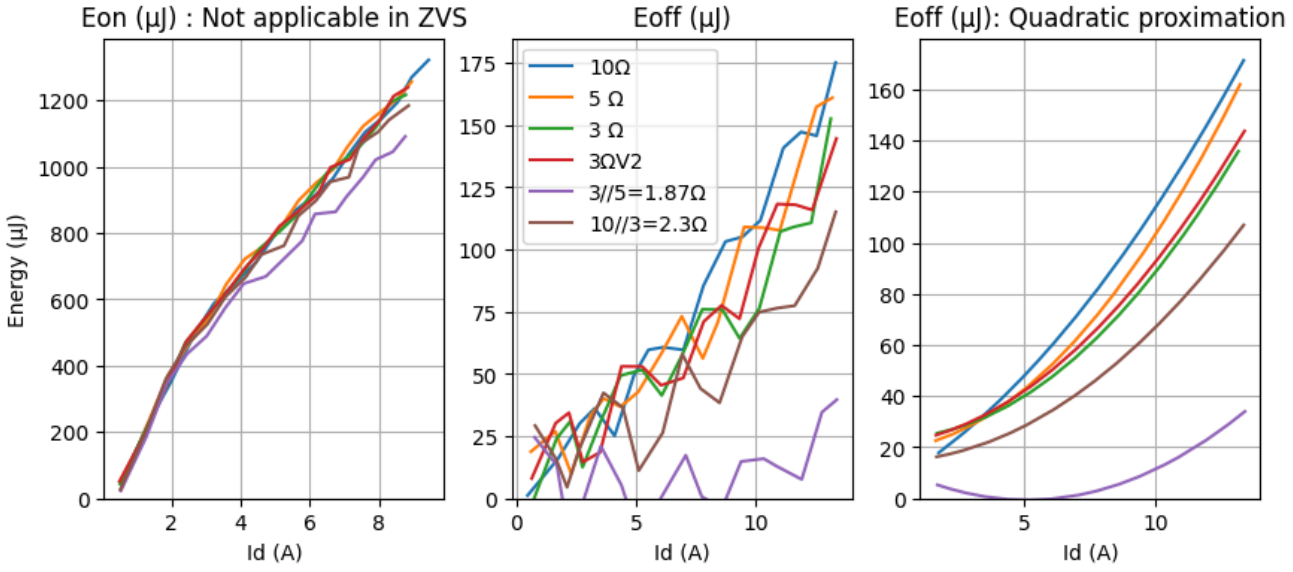


Figure 16 Jumper in the PCB with the 3 resostors (3D)

We followed the same steps as shown in the example above to calculate the Eon/Eoff of each Rg configuration



Rg = 3 Ω is a good compromise between reducing the switching energy and avoiding oscillation, we will conserve this value for this project

Gate current

n this test, we measured the gate current using TCP202, so a small loop wire is added to the current path to insert the current probe (inductance)

The test was done in low voltage condition:

- Vin = 48.03V
- Vout = 8.03V
- Freq = 150kHz (Max freq of the DC-DC)

This test is done with the old primary mosfet (before updating the BOM)

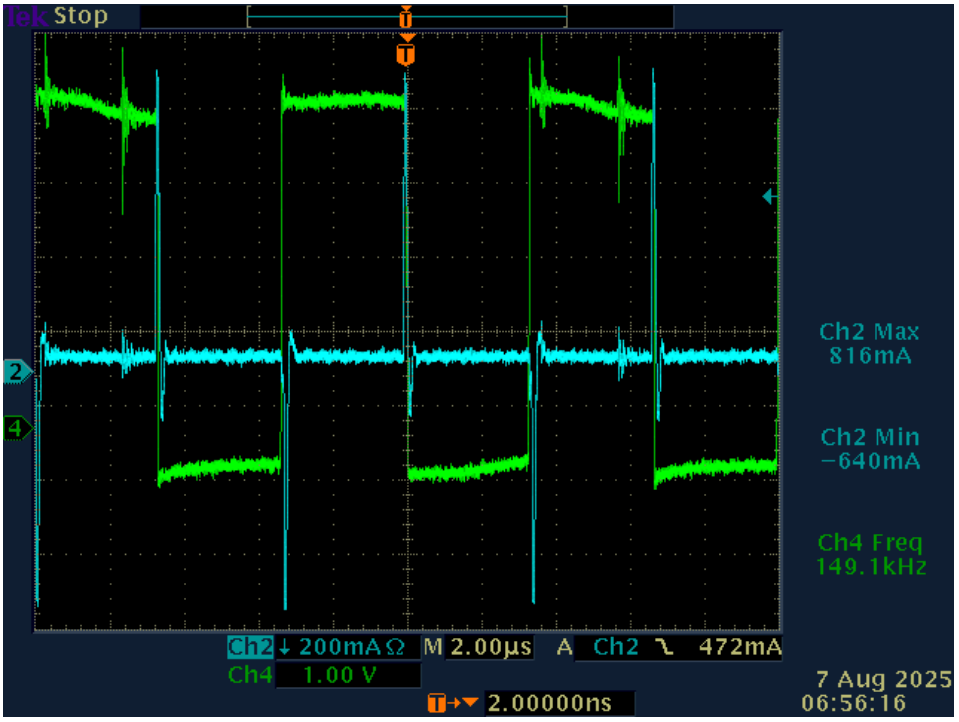
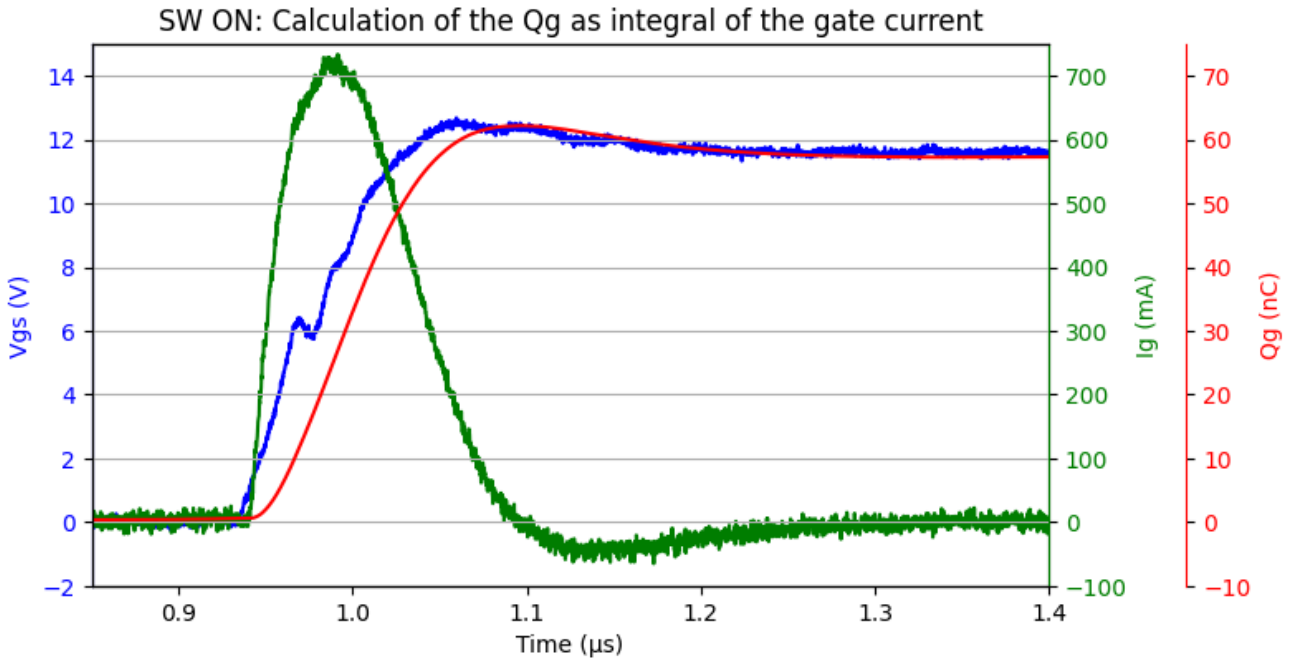


Figure 17 Primary-Low side gate current/voltage

Pre-Processing of the oscilloscope raw data



We canestimate the R_{gTot}
Let's consider that so inductance is present the the gate loop.

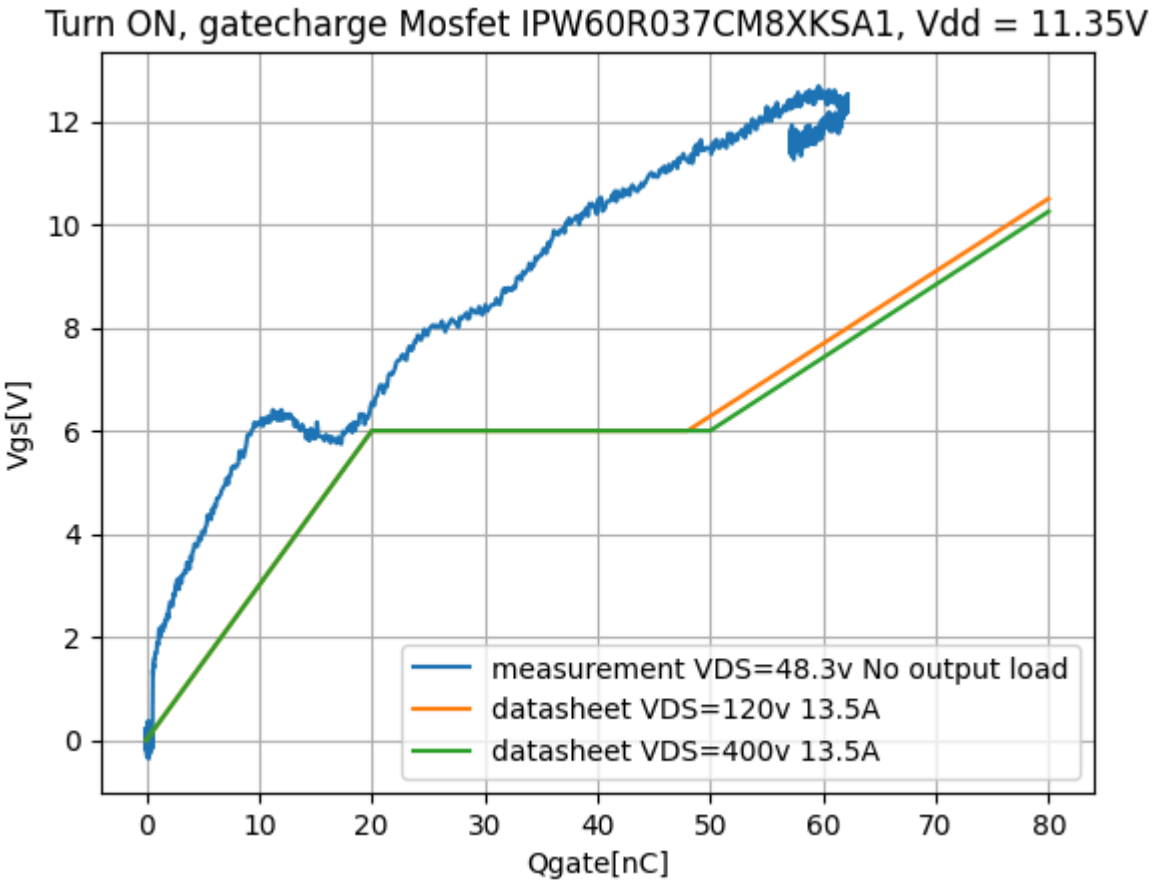
$$
\begin{aligned}
V_{DRV} &= 12 \\
V_{gsTpeak} &= 6 \\
I_{gTpeak} &= 0.730 \\
R_{gTot} &= \frac{V_{DRV} - V_{gsTpeak}}{I_{gTpeak}} = \frac{12 - 6}{0.730} \qquad = 8.219 \ \ (\text{ohm})
\end{aligned}
$$

Theorical value :

$$
\begin{aligned}
R_{gMosfet} &= 0.860 \ \ (\Omega \text{ , see IPW60R037P7 datasheet}) \\
R_{driver} &= 5 \ \ (\Omega \text{ , see UCC21520Q datasheet}) \\
R_{ext} &= 3 \ \ (\Omega \text{ , design}) \\
R_{tot} &= R_{gMosfet} + R_{driver} + R_{ext} = 0.860 + 5 + 3 \qquad = 8.860 \ \ (\Omega)
\end{aligned}
$$

The results are very close

MOSFET: Miller plateau



The Miller plateau is not very pronounced because the SW-ON is in ZVS mode and at low voltage.

Secondary driver

Reminder of the secondary-side synchronous rectifier driver

the master ic : NCP43080DDR2G

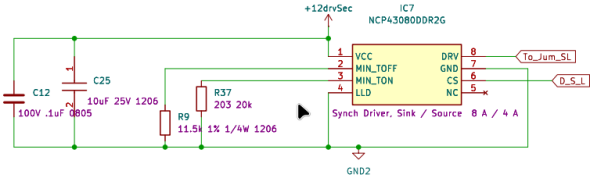


Figure 18 The driver NCP43080DDR2G

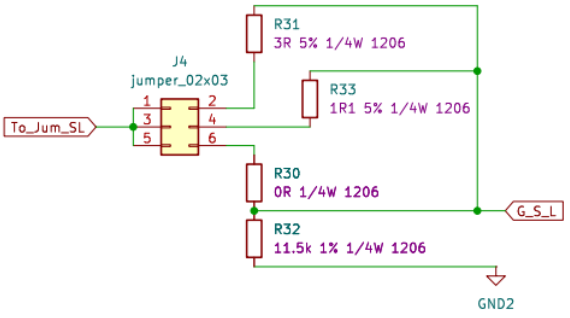


Figure 19 Jumper to shose Rg from 0, 1, 3 ohm

The controller presents a limitation, its conduction time depends heavily on the load; see the test below.

Test conditions:

- Vin = 393.5 v
- Vout = 48.11 v

Oscilloscope:

- CH1 : Vds low side primary x 100
- CH2 : Vgs low side secondary: driver output
- CH4 : Vds low side secondary x 10

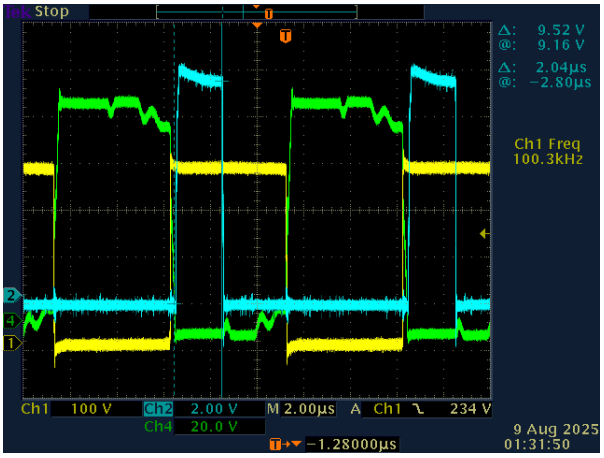


Figure 20 no load

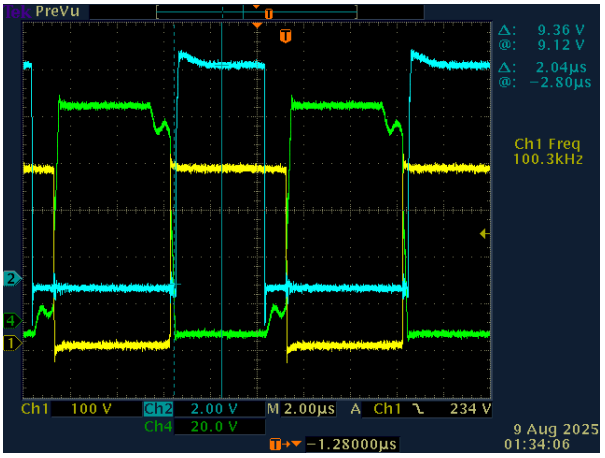


Figure 21 load1 52.1 ohm

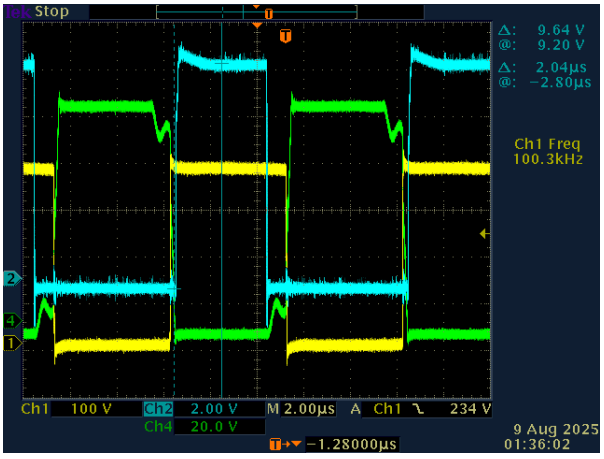


Figure 22 load2 36 ohm

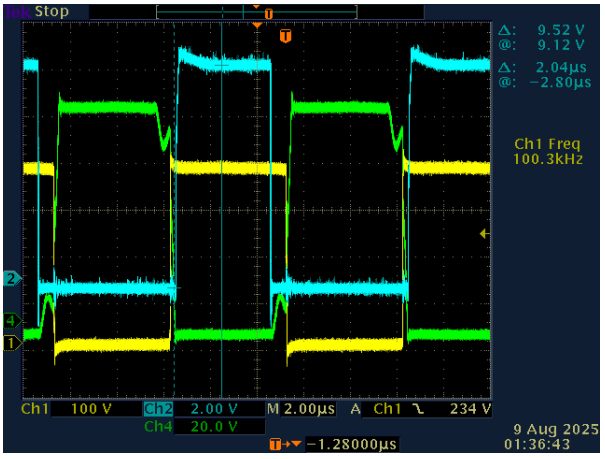


Figure 23 load3 3 24.5 ohm

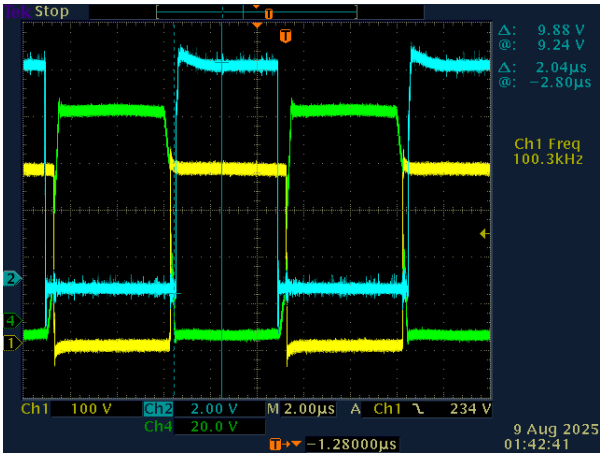


Figure 24 load4 12.8 ohm

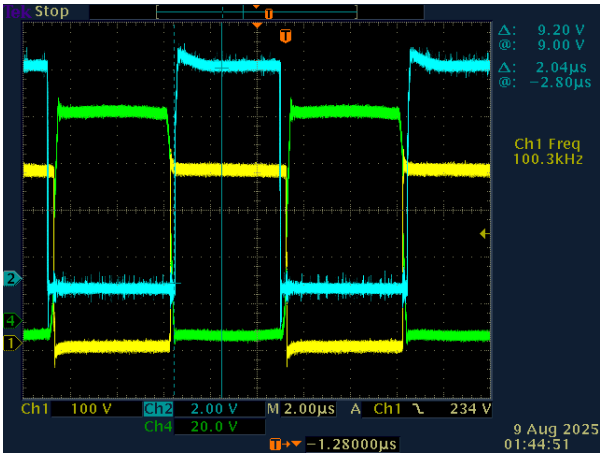


Figure 25 load5 9.2 ohm

Grouping of all test data :

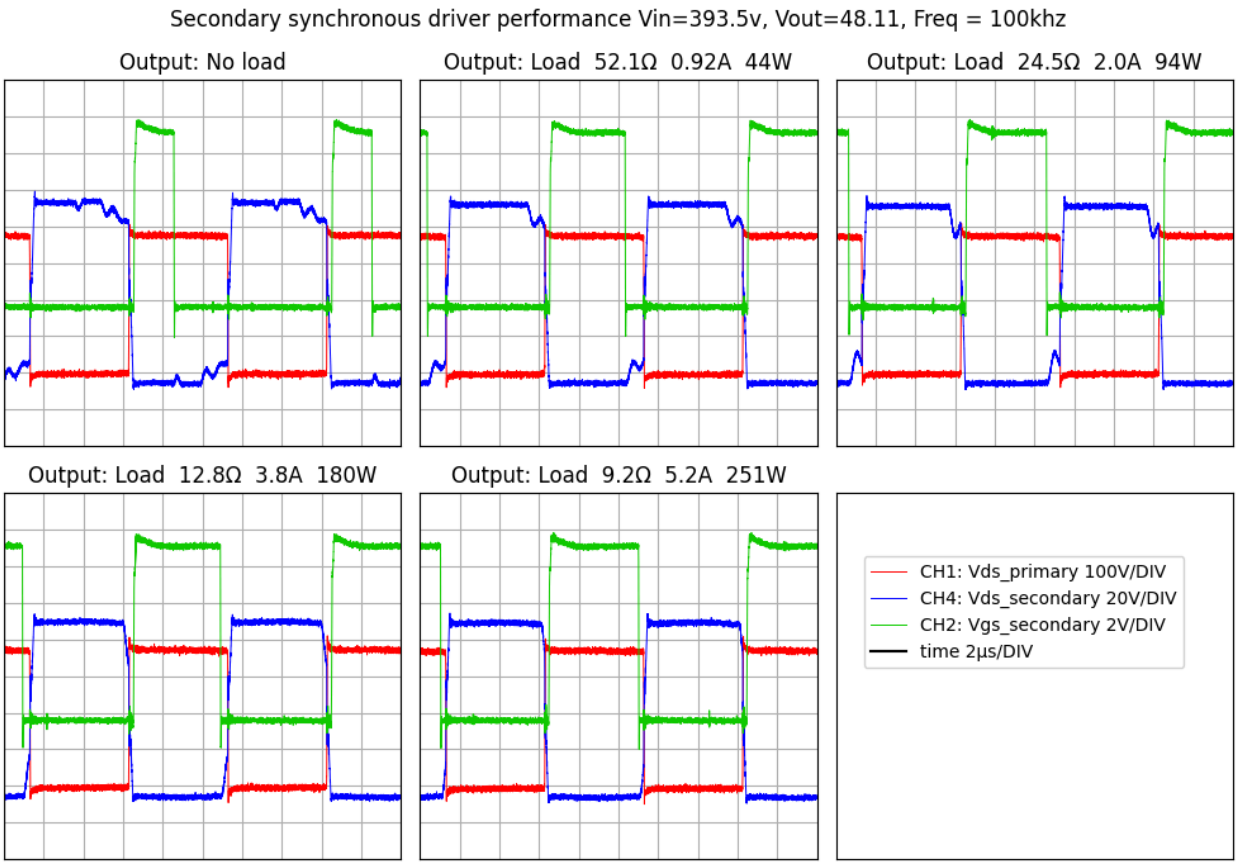


Figure 26 PreProcessing of the test results

Annexes

Carat of the big Lm

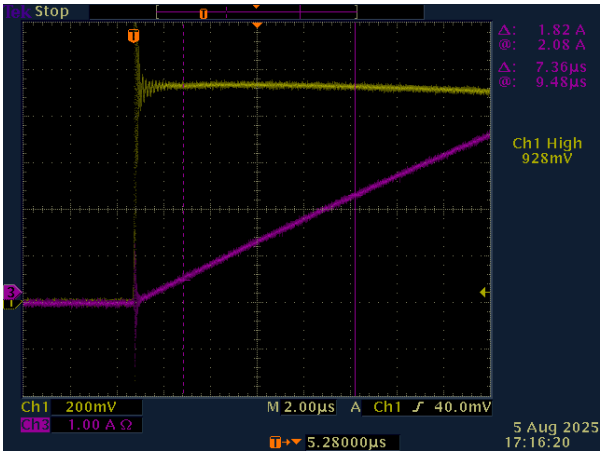


Figure 27 Current

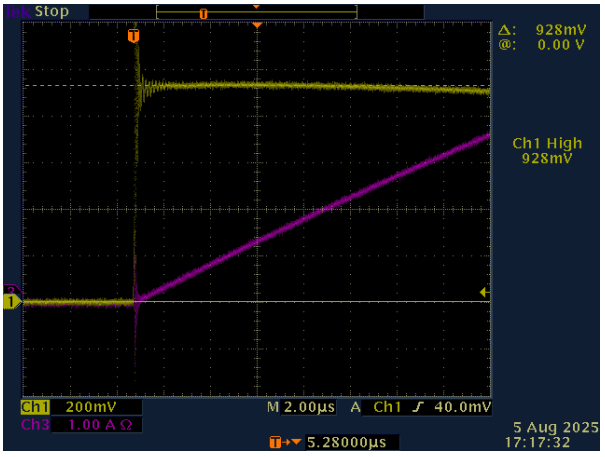


Figure 28 Voltage

We used the same double-pulse setup to characterize the large 16-turn inductor.

See the oscilloscope above (CH1 is and isolated x 100 prob)

$$N = 16 \ \ (\text{turns})$$
$$\Delta_I = 1.820 \ \ (\text{A})$$
$$\Delta_t = 7.360 \ \ (\mu\text{s})$$
$$V_L = 928 \cdot 100 \cdot 1 \times 10^{-3} \qquad = 92.800 \ \ (\text{V})$$

Since $V_L = L\frac{di}{dt}$

$$L = V_L \cdot \left(\frac{\Delta_t}{\Delta_I}\right) = 92.800 \cdot \left(\frac{7.360}{1.820}\right) \qquad = 375.279 \ \ (\mu\text{H})$$
$$\text{AL} = 1 \times 10^3 \cdot \frac{L}{(N)^2} = 1 \times 10^3 \cdot \frac{375.279}{(16)^2} \qquad = 1465.934 \ \ (\text{nH/turn2})$$

Quick prototyping of rogowski coil

During this project, we experimented with several methods to produce a prototype of a Rogowski coil, The idea is to prototype a very simple Rogowski coil and perform the integration via signal preprocessing.

the produced signal is:
$e = k.\, di/dt$

So a preprocessing integral is necessary and an empirical estimation of the coed k

Below some image of the prototyped coils:

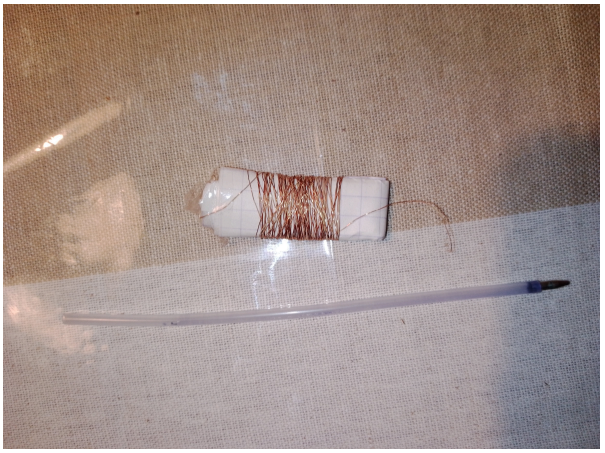


Figure 29

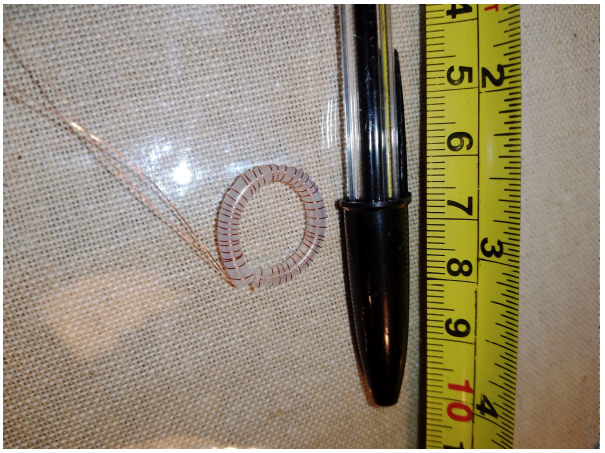


Figure 30

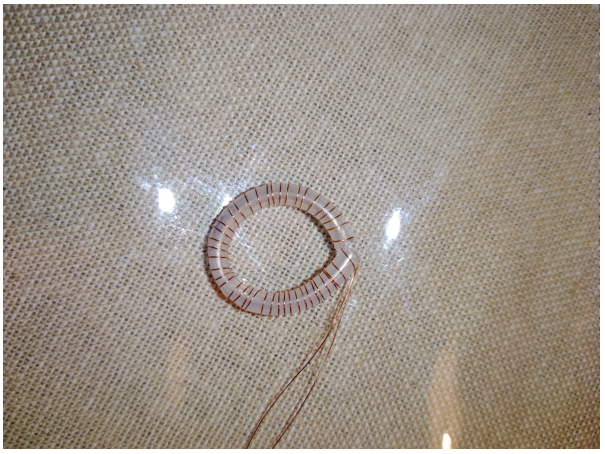


Figure 31

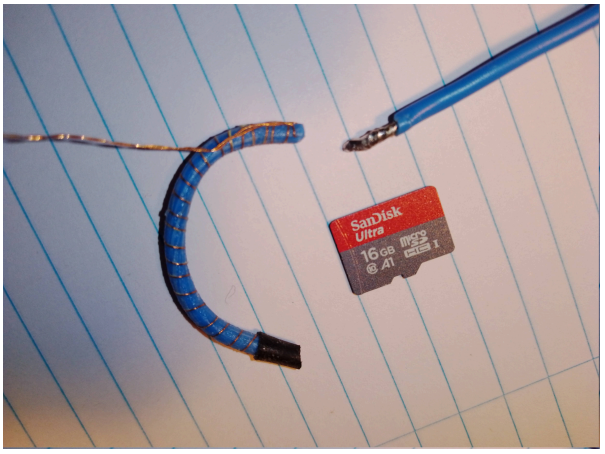


Figure 32

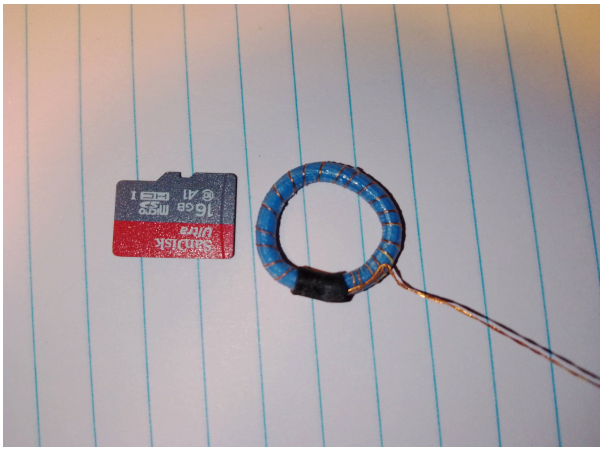


Figure 33

Figure 34

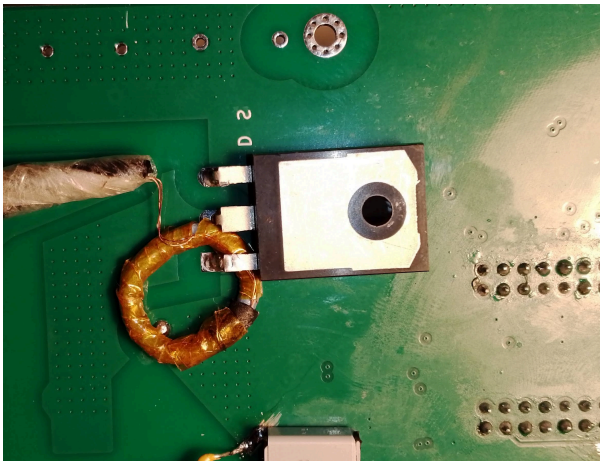


Figure 35

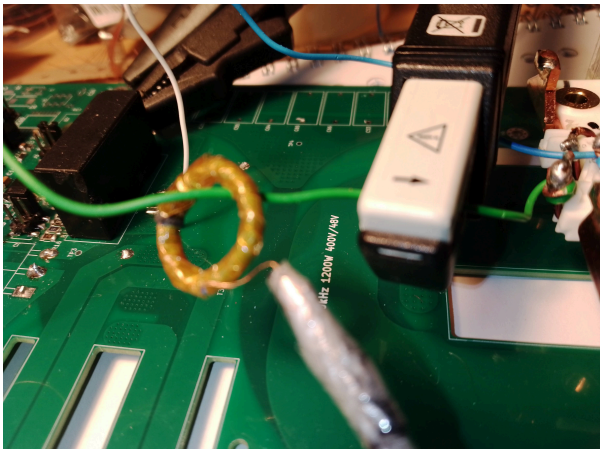


Figure 36

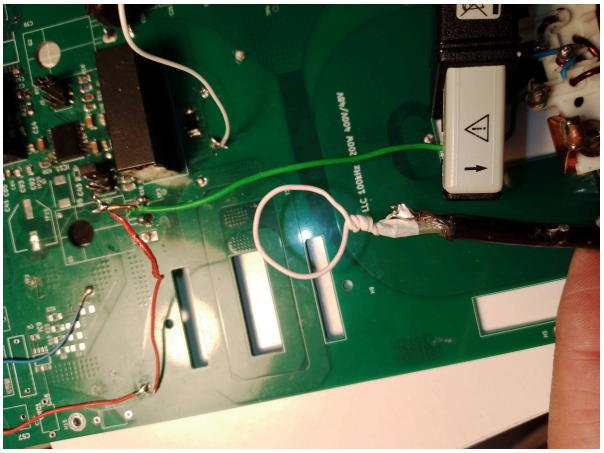


Figure 37

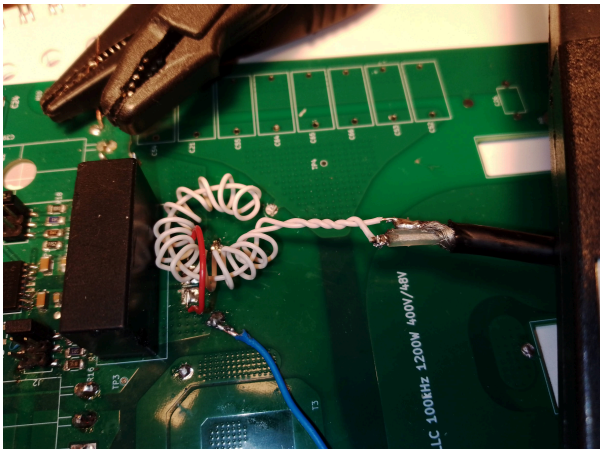


Figure 38

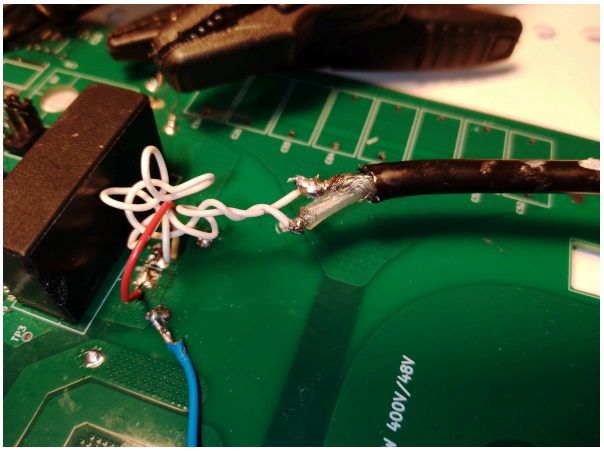


Figure 39

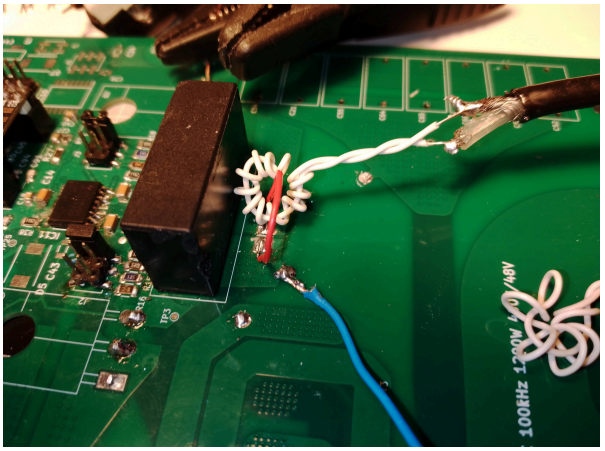


Figure 40

The proprocessing

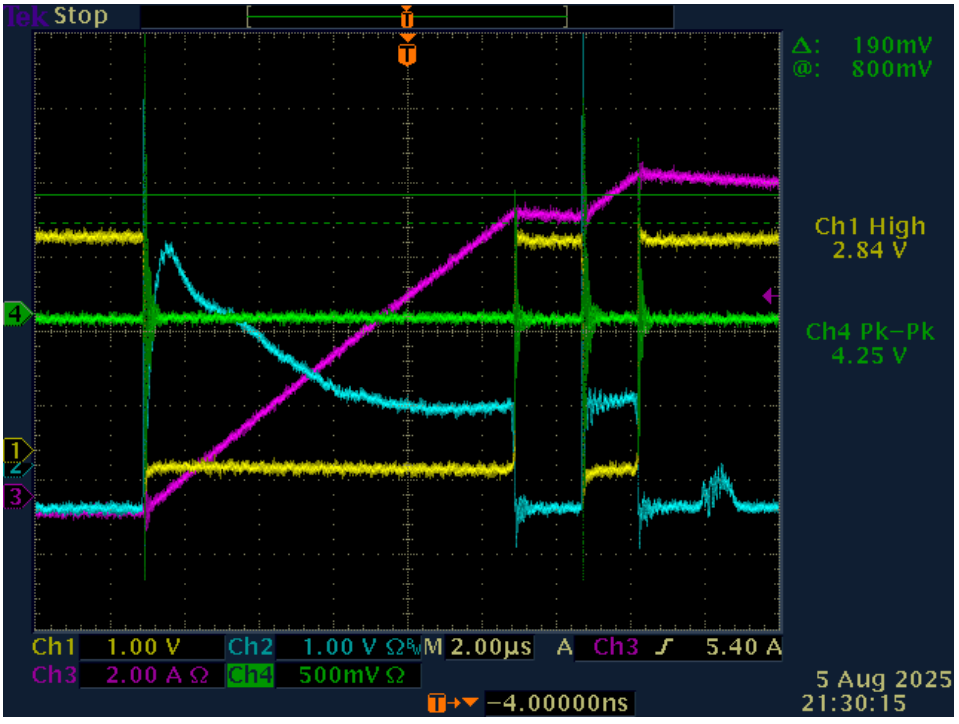
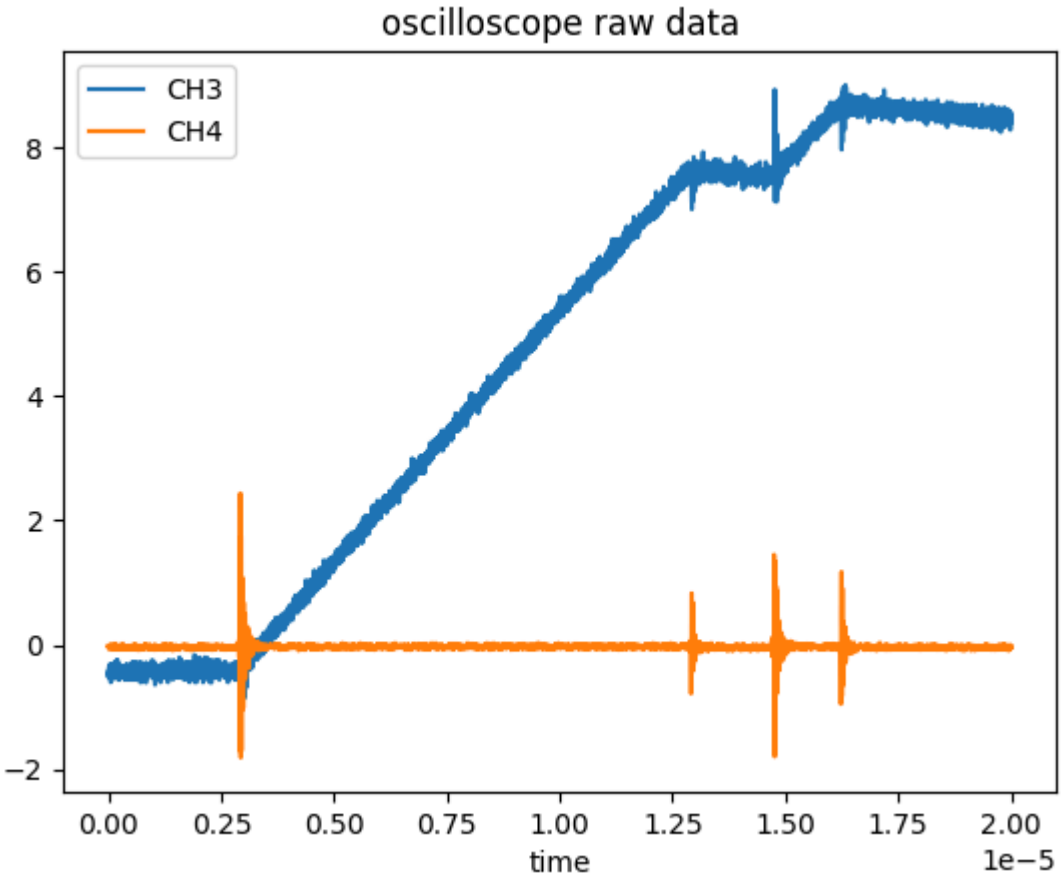
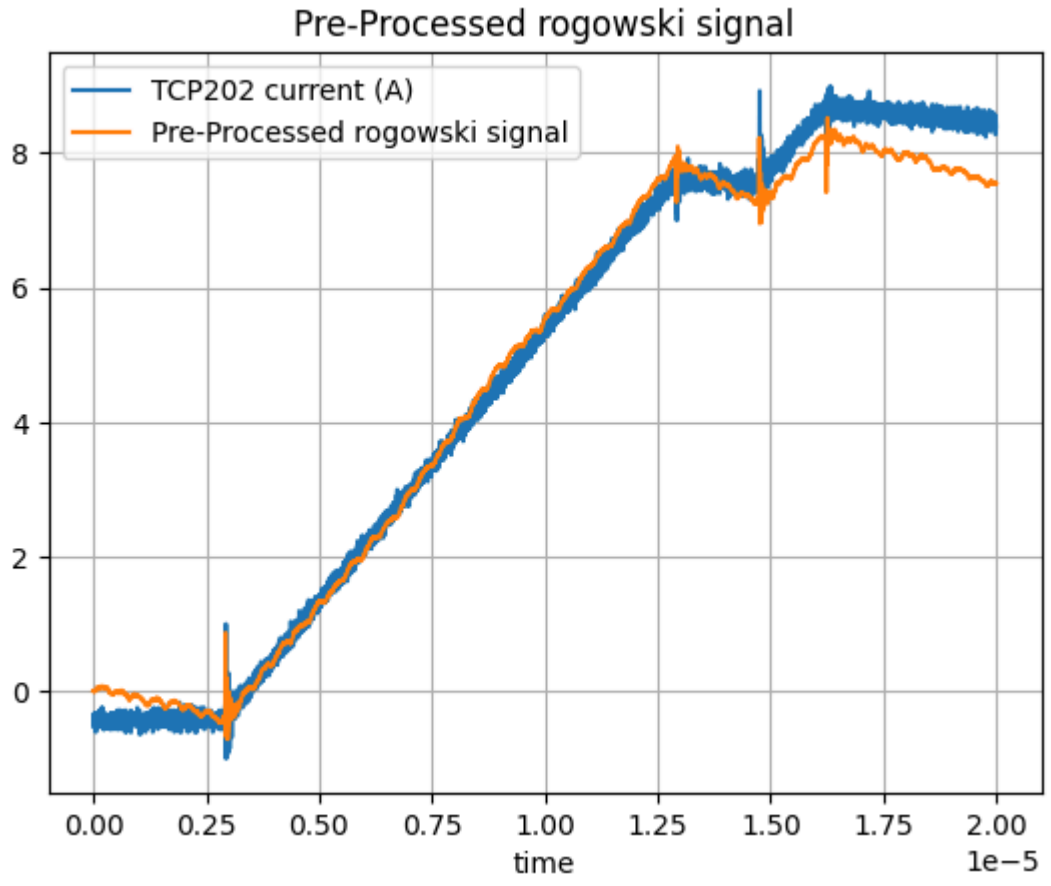


Figure 41 Example of measurement

- CH3 is TCP202 360µH current
- CH4 Rogowski coil voltage (therminal 50ohm)





Preprocessed rogowski signal is very good to detect fast change of the current but bad in low frequency area

Double pulse code STM32f103

Simple double pulse code

```
#include "stm32f1xx.h"

#define CPU_FREQ 72000000UL
#define PA9_SET  (1u << 9)
#define PA9_RST  (1u << (9 + 16))
#define PA9_BIT  (1u << 9)

static inline void dwt_start(void) {
  CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
  DWT->CYCCNT = 0;
  DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}


void setup(void) {
  // Enable GPIOA clock
  RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

  // PA9 = General purpose push-pull output, 50 MHz (CRH bits [7:4])
  GPIOA->CRH &= ~(0xFu << 4);
  GPIOA->CRH |=  (0x3u << 4);   // MODE9=11 (50MHz), CNF9=00 (GP PP)
```

```
  // Ensure USART1 doesn't grab PA9 (we don't enable it)
  RCC->APB2ENR &= ~RCC_APB2ENR_USART1EN;

  dwt_start();

  // Quick self-check: force LOW and verify ODR reflects it
  GPIOA->BSRR = PA9_RST;
  if (GPIOA->ODR & PA9_BIT) { for (;;){ /* trap if config failed */ } }
}

void loop(void) {
  uint32_t ton1  = 2; // us
  uint32_t toff1 = 1; // us
  uint32_t ton2  = 1; // us
  uint32_t toff2 = 1; // us

  GPIOA->BSRR = PA9_SET;
  delayMicroseconds(ton1);

  GPIOA->BSRR = PA9_RST;
  delayMicroseconds(toff1);

  GPIOA->BSRR = PA9_SET;
  delayMicroseconds(ton2);

  GPIOA->BSRR = PA9_RST;
  delayMicroseconds(toff2);
}
```

Train of double pulse

```
#include "stm32f1xx.h"

// --- PA9 macros ---
#define PA9_SET   (1u << 9)
#define PA9_RST   (1u << (9 + 16))

// --- PB14 macros ---
#define PB14_SET  (1u << 14)
#define PB14_RST  (1u << (14 + 16))


#define _Ena_drv PB10
#define _Dis_drv PB12

void setup() {
// Force PB12 HIGH in output data register BEFORE pinMode changes the mode
// digitalWrite(_Dis_drv, HIGH);     // preload ODR = 1 (float high in
open-drain mode)
digitalWrite(_Dis_drv, LOW);
pinMode(_Dis_drv, OUTPUT_OPEN_DRAIN);
digitalWrite(_Dis_drv, LOW); // redundancy
digitalWrite( _Ena_drv, HIGH);
pinMode(_Dis_drv, OUTPUT);
digitalWrite( _Ena_drv, HIGH);
```

```
/////// PA9 & PB14 as Fast output ///////////
  // Enable GPIOA + GPIOB clocks
  RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPBEN;

  // --- PA9 = 50 MHz push-pull output ---
  GPIOA->CRH &= ~(0xF << 4);   // clear config bits
  GPIOA->CRH |=  (0x3 << 4);   // MODE9=11 (50MHz), CNF9=00
  GPIOA->BSRR = PA9_RST;       // start LOW

  // --- PB14 = 50 MHz push-pull output ---
  GPIOB->CRH &= ~(0xF << 24);  // PB14 config bits
  GPIOB->CRH |=  (0x3 << 24);  // MODE14=11, CNF14=00
  GPIOB->BSRR = PB14_RST;      // start LOW



}




__attribute__((always_inline))
static inline void delayNs(uint32_t tns)
{
    /*
    target => oscilo
    6000ns => 5986ns
    1000ns => 985.3ns
    */

    uint32_t iterations = (tns * 72) / (6.08*1000);

    for (uint32_t i = 0; i < iterations; i++) {
        __asm__ __volatile__("nop" ::: "memory");
    }
}


void loop() {


////////////// ENABLE DRIVER
delayMicroseconds(10);
digitalWrite(_Dis_drv, HIGH); // redundancy
digitalWrite( _Ena_drv, LOW);
delayMicroseconds(10);
digitalWrite( _Ena_drv, HIGH);


////////// double pulse waveform on / off / on /off

  uint32_t ton  = 1000; // ns
```

```
  uint32_t toff = 800; // ns
  uint32_t N  = 17; // n of pulses

  for (int i = 0; i<N; i++){
  GPIOA->BSRR = PA9_SET;
  delayNs(ton);
  GPIOA->BSRR = PA9_RST;
  delayNs(toff);
  }


  while(1);


}
```